

Contexto, Objetivo e Hipóteses do Projeto

- Contexto: . O setor bancário lida com um volume massivo de dados de clientes, mas transformar esses dados em depósitos contratados exige precisão cirúrgica. Atualmente, as campanhas de marketing direto por telefone enfrentam baixas taxas de conversão (apenas 11,7% no dataset analisado), resultando em desperdício de tempo e recursos. Este projeto propõe a criação de uma aplicação interativa via Streamlit que utiliza IA para classificar a propensão de compra de cada cliente. - Objetivos: . Objetivo Geral: Desenvolver e implantar uma aplicação preditiva que sirva de interface para a equipe de marketing, permitindo o ranqueamento de leads e a simulação de cenários de conversão em tempo real. . Objetivos Específicos: * Operacionalização da Triagem (Ranking): Permitir que o usuário do Streamlit identifique rapidamente o "Top 10%" de clientes * Interpretabilidade Local: Integrar a tecnologia SHAP à aplicação para que o gerente de contas entenda exatamente quais fatores (como saldo ou idade) estão influenciando o score de um cliente específico; * Prevenção de Evasão/Fadiga: Utilizar o modelo para sinalizar quando um cliente atingiu um nível crítico de contatos (ex: campaign > 20), onde a probabilidade de conversão cai drasticamente. - Hipóteses: . H1 - Histórico de Campanha: O sucesso em abordagens anteriores (poutcome) é o preditor mais forte de novas conversões, indicando uma fidelidade ao produto; . H2 - Liquidez e Solvência: Clientes com saldos anuais positivos (balance) e sem inadimplência (default) apresentam maior propensão ao investimento em depósitos a prazo; . H3 - Barreiras de Crédito: A existência de um empréstimo habitacional (housing) atua como uma força redutora na probabilidade de conversão, possivelmente devido ao comprometimento da renda mensal; . H4 - Perfil Demográfico Sênior: Indivíduos em faixas etárias mais elevadas (60-80 anos), mesmo se não aposentados, demonstram maior interesse em produtos de baixo risco como o depósito a prazo; . H5 - Eficácia do Canal: Contatos realizados via celular apresentam taxas de conversão superiores às de telefones fixos, refletindo a modernização dos canais de comunicação com o cliente.

Carregamento de Dados e Entendimento Inicial dos Dados

- Esta seção apresenta o carregamento do dataset e uma verificação inicial de sua estrutura, incluindo dimensionalidade, tipos de variáveis e possíveis problemas evidentes, como valores faltantes ou inconsistências. - Os dados referem-se a campanhas de marketing direto de uma instituição bancária portuguesa. As campanhas de marketing baseavam-se em chamadas telefônicas. Frequentemente, era necessário mais do que um contato com o mesmo cliente para apurar se o produto (depósito a prazo bancário) seria contratado ("sim") ou não ("não").

```
In [4]: # =====  
# 1. Padrão e Visualização  
# =====  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import warnings  
  
warnings.filterwarnings("ignore")  
# Para visualização do SHAP no notebook  
import shap  
  
# =====  
# 2. Estatística e Matemática  
# =====  
from scipy import stats  
from scipy.stats import chi2_contingency  
  
# =====  
# 3. Pré-processamento e Transformação  
# =====  
from sklearn.preprocessing import (
```

```

        OneHotEncoder,
        StandardScaler,
    )
    from sklearn.compose import ColumnTransformer
    from sklearn.impute import SimpleImputer
    from sklearn.pipeline import Pipeline
    from sklearn.utils.class_weight import compute_sample_weight
    from tempfile import mkdtemp

    # =====
    # 4. Modelagem e Seleção
    # =====
    from sklearn.experimental import enable_halving_search_cv
    from sklearn.model_selection import train_test_split, HalvingRandomSearchCV
    from sklearn.feature_selection import SelectKBest, mutual_info_classif

    # =====
    # 5. Algoritmos de Classificação
    # =====
    from sklearn.dummy import DummyClassifier
    from sklearn.linear_model import LogisticRegression
    from sklearn.ensemble import RandomForestClassifier
    from xgboost import XGBClassifier
    from lightgbm import LGBMClassifier

    # =====
    # 6. Métricas de Avaliação
    # =====
    from sklearn.metrics import (
        roc_auc_score,
        roc_curve,
        average_precision_score,
        confusion_matrix,
        classification_report,
        ConfusionMatrixDisplay,
        brier_score_loss,
        precision_recall_curve
    )

    from sklearn.calibration import calibration_curve

    # =====
    # 7. Produção/Exportação
    # =====
    import joblib

    # =====
    # 8. Repositório
    # =====
    from ucimlrepo import fetch_ucirepo

```

```

In [5]: # Dataset Banking Market (UCI)
bank_marketing = fetch_ucirepo(id=222)

X = bank_marketing.data.features
y = bank_marketing.data.targets

```

```
df = pd.concat([X,y], axis = 1)
df.head()
```

```
Out[5]:
```

	age	job	marital	education	default	balance	housing	loan	contact	day_of
0	58	management	married	tertiary	no	2143	yes	no	NaN	
1	44	technician	single	secondary	no	29	yes	no	NaN	
2	33	entrepreneur	married	secondary	no	2	yes	yes	NaN	
3	47	blue-collar	married	NaN	no	1506	yes	no	NaN	
4	33	NaN	single	NaN	no	1	no	no	NaN	

- Legenda de classe

Variable Role Type Description -----
 ----- age Feature Integer Age of the individual. job Feature Categorical Occupation: 'admin.', 'blue-collar', 'entrepreneur', 'housemaid', 'management', 'retired', 'self-employed', 'services', 'student', 'technician', 'unemployed', 'unknown'. marital Feature Categorical Marital Status: 'divorced', 'married', 'single'. education Feature Categorical Education Level: 'primary', 'secondary', 'terciary'. default Feature Binary has credit in default? balance Feature Integer average yearly balance (euros). housing Feature Binary has housing loan? loan Feature Binary has personal loan? contact Feature(?) Categorical contact communication type: 'cellular', 'telephone'. day_of_week Feature Date last contact day of the week. month Feature Date last contact month of year: 'jan', 'feb', 'mar', ..., 'nov', 'dec'. duration Feature(?) Integer last contact duration in seconds. campaign Feature Integer number of contacts during this campaign for this client. pdays Feature Integer days since last contact in a previous campaign: -1 means not previously contacted. previous Feature Integer number of contacts before this campaign for this client. poutcome Feature(?) Categorical outcome of the previous marketing campaign: 'failure', 'nonexistent', 'success'. y Target Binary has the client subscribed a term deposit? -----

- Entendimento inicial

```
In [6]: rows, columns = df.shape
print(f"rows: {rows}, columns: {columns}")
```

```
rows: 45211, columns: 17
```

```
In [7]: df.dtypes
```

```
Out[7]: age          int64
        job          object
        marital      object
        education    object
        default      object
        balance      int64
        housing      object
        loan         object
        contact      object
        day_of_week  int64
        month        object
        duration     int64
        campaign     int64
        pdays        int64
        previous     int64
        poutcome     object
        y            object
        dtype: object
```

```
In [8]: print(f"Duplicated values: {df.duplicated().sum()}")
```

Duplicated values: 0

```
In [9]: # Valores faltantes
        print("Null values")
        print("-"*30)
        df.isna().sum()
```

Null values

```
-----
Out[9]: age          0
        job          288
        marital      0
        education    1857
        default      0
        balance      0
        housing      0
        loan         0
        contact      13020
        day_of_week  0
        month        0
        duration     0
        campaign     0
        pdays        0
        previous     0
        poutcome     36959
        y            0
        dtype: int64
```

```
In [10]: # A variável independente "y" : yes or no (binária)
        # O quanto de cada classe (em %) é encontrado no dataset
        df['y'].value_counts(normalize = True)*100
```

```
Out[10]: y
         no      88.30152
         yes    11.69848
         Name: proportion, dtype: float64
```

- Identificação de potenciais problemas

- Data leakage: . Duration gera dataleakage, já que se utiliza o número de contatos feitos ao cliente, ou seja, aquele que foi "convencido" pela propaganda (observações do futuro usadas no passado); . As variáveis contact e poutcome apresentam ausência não aleatória, sendo preenchidas apenas quando há registro de tentativa de contato ou de campanha anterior, respectivamente. Dessa forma, o padrão de missingness codifica o estado do processo de campanha, e não uma característica intrínseca do indivíduo, o que pode introduzir viés operacional ou caracterizar vazamento conceitual de informação. - Desbalanceamento moderado de dados (10% para a classe minoritária de target). - Tipos errados (datas como string): . Já existe no dataset mês e dia; . "default", "housing", "loan" precisam ser transformadas via map (0/1). - Variáveis com significado especial: . month e day_of_week: Extremamente importante. Diferente de datasets de trânsito e correlatos, este dificilmente tem forte padrão temporal ,portanto, não deve ser alvo de transformação via seno/cosseno, transformá-los diretamente em categorias traria alta dimensionalidade. Sendo assim, um agrupamento semântico é uma alternativa. - Variáveis derivadas do target (y): . Não há Função direta de y ou construída ou um componente matemático do target.

Análise Exploratória

- A análise exploratória teve como objetivo compreender a estrutura do conjunto de dados, identificar padrões relevantes, avaliar limitações informacionais e orientar decisões metodológicas nas etapas subsequentes de modelagem. Dada a natureza aplicada do problema — predição de subscrição de um depósito a prazo — a EDA foi conduzida com foco não apenas estatístico, mas também operacional e interpretativo. - Distribuição das variáveis numéricas: . As variáveis numéricas apresentaram distribuições heterogêneas, frequentemente assimétricas e com presença de caudas longas. Em especial, variáveis relacionadas ao histórico de campanha, como pdays, previous e campaign, exibiram padrões não gaussianos e concentrações em valores específicos, refletindo a dinâmica real de contatos comerciais; . As variáveis pdays, previous e campaign apresentaram comportamento predominantemente discreto, com forte concentração de massa em poucos valores inteiros. Visualmente, suas distribuições são caracterizadas por picos pronunciados e rápida queda de frequência, o que dificulta a identificação de caudas no sentido clássico de variáveis contínuas; . Variáveis como balance e age apresentaram maior dispersão e assimetria moderada, indicando que fatores socioeconômicos podem contribuir para a decisão de subscrição, ainda que de forma não determinística. - Variáveis categóricas e padrões comportamentais: . As variáveis categóricas revelaram padrões relevantes associados ao comportamento do cliente e à forma de abordagem da campanha. Variáveis como job, education, contact e poutcome apresentaram distribuições desiguais entre as classes, sugerindo relações não lineares e dependentes de contexto; . Em particular, o resultado de campanhas anteriores (poutcome) mostrou-se altamente informativo já na análise exploratória, indicando que clientes com histórico positivo tendem a apresentar maior propensão à subscrição em novas abordagens. Por outro lado, clientes sem histórico prévio formam um grupo mais heterogêneo e difícil de caracterizar; . A análise das frequências relativas evidenciou que a conversão não está associada a perfis extremos, mas sim a combinações específicas de características, reforçando a necessidade de modelos capazes de capturar interações complexas entre variáveis. - Transformações: . Durante a análise exploratória, as variáveis temporais foram reagrupadas no dataset temporário para reduzir granularidade excessiva e facilitar a interpretação dos padrões observados. O dia do mês foi transformado em faixas de dias, evitando análises fragmentadas decorrentes da alta cardinalidade dessa variável. De forma análoga, a variável mês foi agregada em trimestres, permitindo uma visão mais estável de possíveis efeitos sazonais, especialmente considerando a cobertura irregular dos meses no conjunto de dados. - Risco de data leakage e decisões preventivas: . Durante a EDA, foi identificada a variável duration, contact e poutcome como data leakage, segundo teste estatístico (MI).

- Dataset temporário

```
In [11]: df_eda = df.copy()
```

```
In [12]: # Transformando de fato em variáveis binárias
         bin_map = {"yes": 1, "no": 0}

         binaria = ["housing", "loan", "default", "y"]
```

```

for col in binaria:
    df_eda[col] = df_eda[col].map(bin_map)

# Definição das faixas de dias
def group_day_of_month(day):
    if 1 <= day <= 5:
        return "1_5"
    elif 6 <= day <= 10:
        return "6_10"
    elif 11 <= day <= 15:
        return "11_15"
    elif 16 <= day <= 20:
        return "16_20"
    elif 21 <= day <= 25:
        return "21_25"
    else:
        return "26_31"

df_eda["days"] = df_eda["day_of_week"].apply(group_day_of_month)

# Transformar em trimestres
month_map = {
    "jan": "Q1", "feb": "Q1", "mar": "Q1",
    "apr": "Q2", "may": "Q2", "jun": "Q2",
    "jul": "Q3", "aug": "Q3", "sep": "Q3",
    "oct": "Q4", "nov": "Q4", "dec": "Q4"
}

df_eda["quarter"] = df_eda["month"].map(month_map)

# "contact" pode causar Leakage
# Criando uma coluna para testar
df_eda["contact_missing"] = df_eda["contact"].isna().astype(int)

# "poutcome" também pode causar Leakage
# Criando uma coluna para testar
df_eda["poutcome_missing"] = df_eda["poutcome"].isna().astype(int)

```

- Descrição do dataset temporário

```
In [13]: df_eda.describe().round(4)
```

Out[13]:

	age	default	balance	housing	loan	day_of_week	duratio
count	45211.0000	45211.000	45211.0000	45211.0000	45211.0000	45211.0000	45211.000
mean	40.9362	0.018	1362.2721	0.5558	0.1602	15.8064	258.163
std	10.6188	0.133	3044.7658	0.4969	0.3668	8.3225	257.527
min	18.0000	0.000	-8019.0000	0.0000	0.0000	1.0000	0.000
25%	33.0000	0.000	72.0000	0.0000	0.0000	8.0000	103.000
50%	39.0000	0.000	448.0000	1.0000	0.0000	16.0000	180.000
75%	48.0000	0.000	1428.0000	1.0000	0.0000	21.0000	319.000
max	95.0000	1.000	102127.0000	1.0000	1.0000	31.0000	4918.000

- Classificação das variáveis

```
In [14]: cols_num = ["age", "balance", "duration", "campaign", "pdays", "previous"] # dura
cols_bin = ["default", "housing", "loan"] # devidamente transformadas
cols_cat = ["days", "quarter", "job", "marital", "contact", "poutcome"]
cols_semiord = ["education"] # no âmbito da econometria, o impacto de education nã
target = "y" # devidamente transformada
```

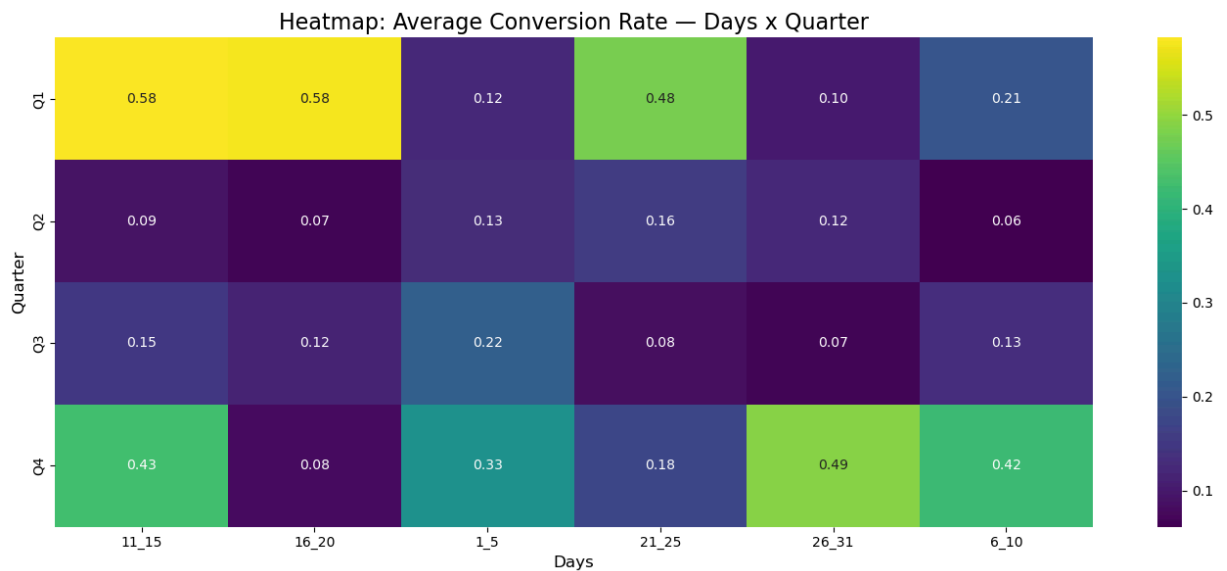
- Heatmap dias x trimestre (taxa média de conversão)

- A taxa média de conversão , em média, é mais alta no primeiro nos dias 11-20 do primeiro trimestre e mais baixa nos dias 6-10 do segundo trimestre.

```
In [15]: pivot = df_eda.pivot_table(
    values="y",
    index="quarter",
    columns="days",
    aggfunc="mean"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    annot=True,
    fmt=".2f",
    cmap="viridis"
)

plt.title("Heatmap: Average Conversion Rate – Days x Quarter", fontsize=16)
plt.xlabel("Days", fontsize=12)
plt.ylabel("Quarter", fontsize=12)
plt.tight_layout()
plt.show()
```



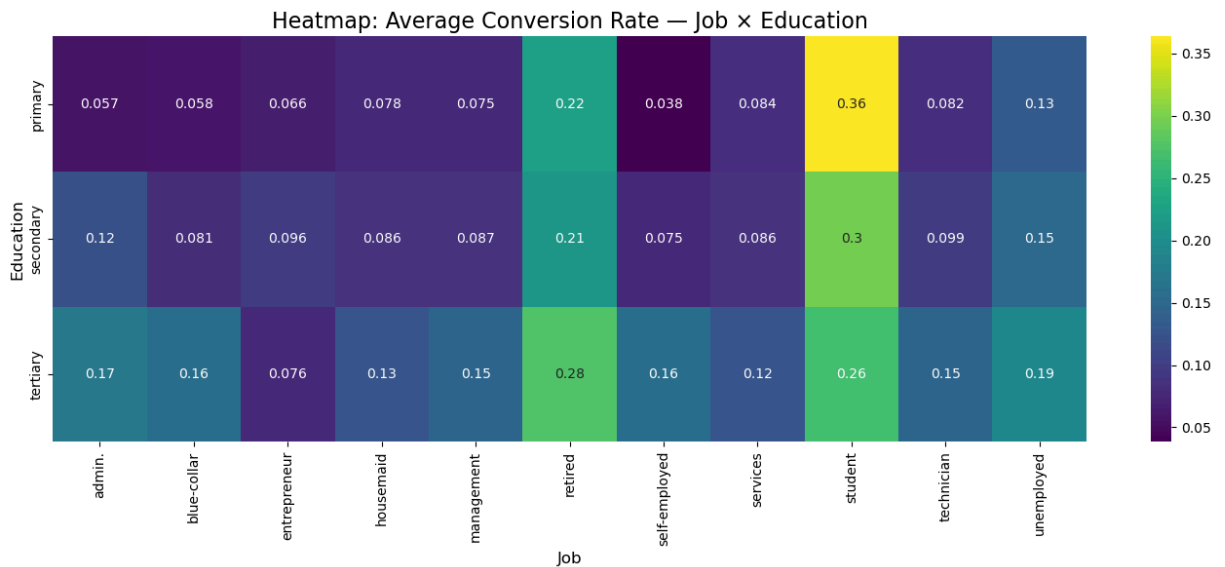
- Heatmap ocupação x educação (taxa média de conversão)

- Os estudantes com educação primária, em média, são aqueles com as maiores taxas médias de conversão, enquanto que os autônomos de mesma escolaridade estão associados as menores taxas.

```
In [16]: pivot = df_eda.pivot_table(
    values="y",
    index="education",
    columns="job",
    aggfunc="mean"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    cmap="viridis",
    annot=True
)

plt.title("Heatmap: Average Conversion Rate — Job x Education", fontsize=16)
plt.xlabel("Job", fontsize=12)
plt.ylabel("Education", fontsize=12)
plt.tight_layout()
plt.show()
```

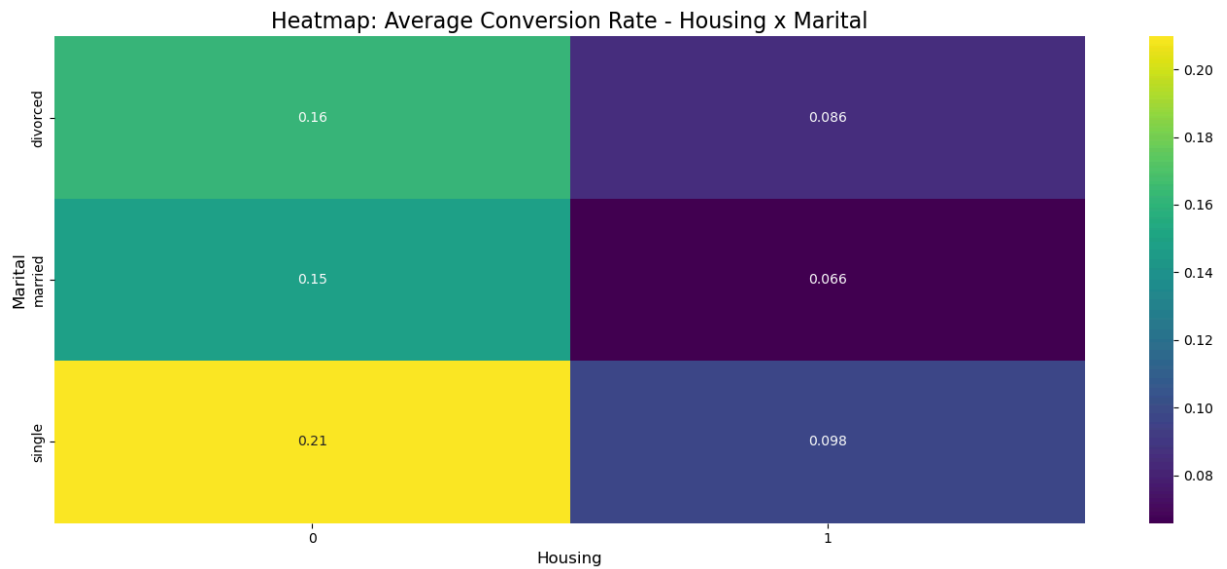
- Heatmap empréstimo habitacional x estado civil (taxa média de conversão)

- Os indivíduos solteiros e sem empréstimo habitacional, em média, apresentam taxas de conversão (média) maiores. Já os casados, com empréstimo habitacional estão associados a taxas menores.

```
In [17]: pivot = df_eda.pivot_table(
    values="y",
    index="marital",
    columns="housing",
    aggfunc="mean"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    cmap="viridis",
    annot=True
)

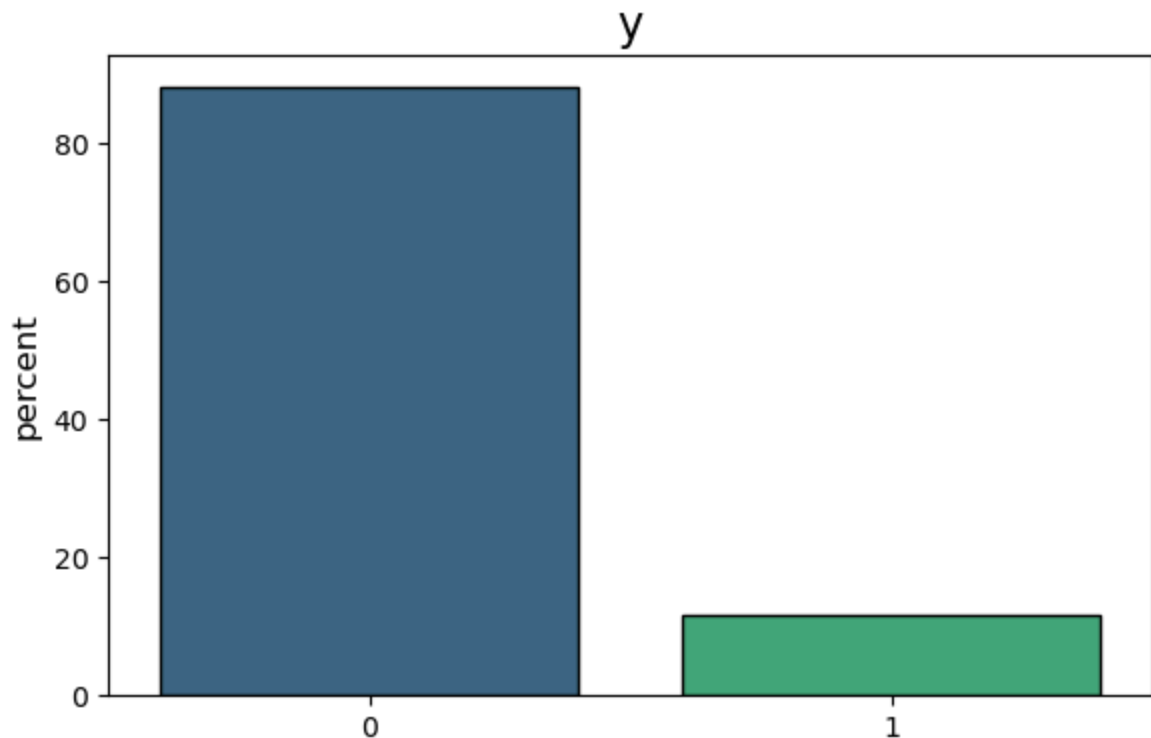
plt.title("Heatmap: Average Conversion Rate - Housing x Marital", fontsize=16)
plt.xlabel("Housing", fontsize=12)
plt.ylabel("Marital", fontsize=12)
plt.tight_layout()
plt.show()
```



- Distribuição da variável target

- A variável target sofre de desbalanceamento moderado.

```
In [18]: plt.figure(figsize=(6,4))
sns.countplot(
    data=df_eda,
    x="y",
    stat="percent",
    palette="viridis",
    edgecolor="black"
)
plt.title("y", fontsize=16)
plt.xlabel('', fontsize=12)
plt.ylabel("percent", fontsize=12)
plt.tight_layout()
plt.show();
```



- Distribuição das variáveis numéricas

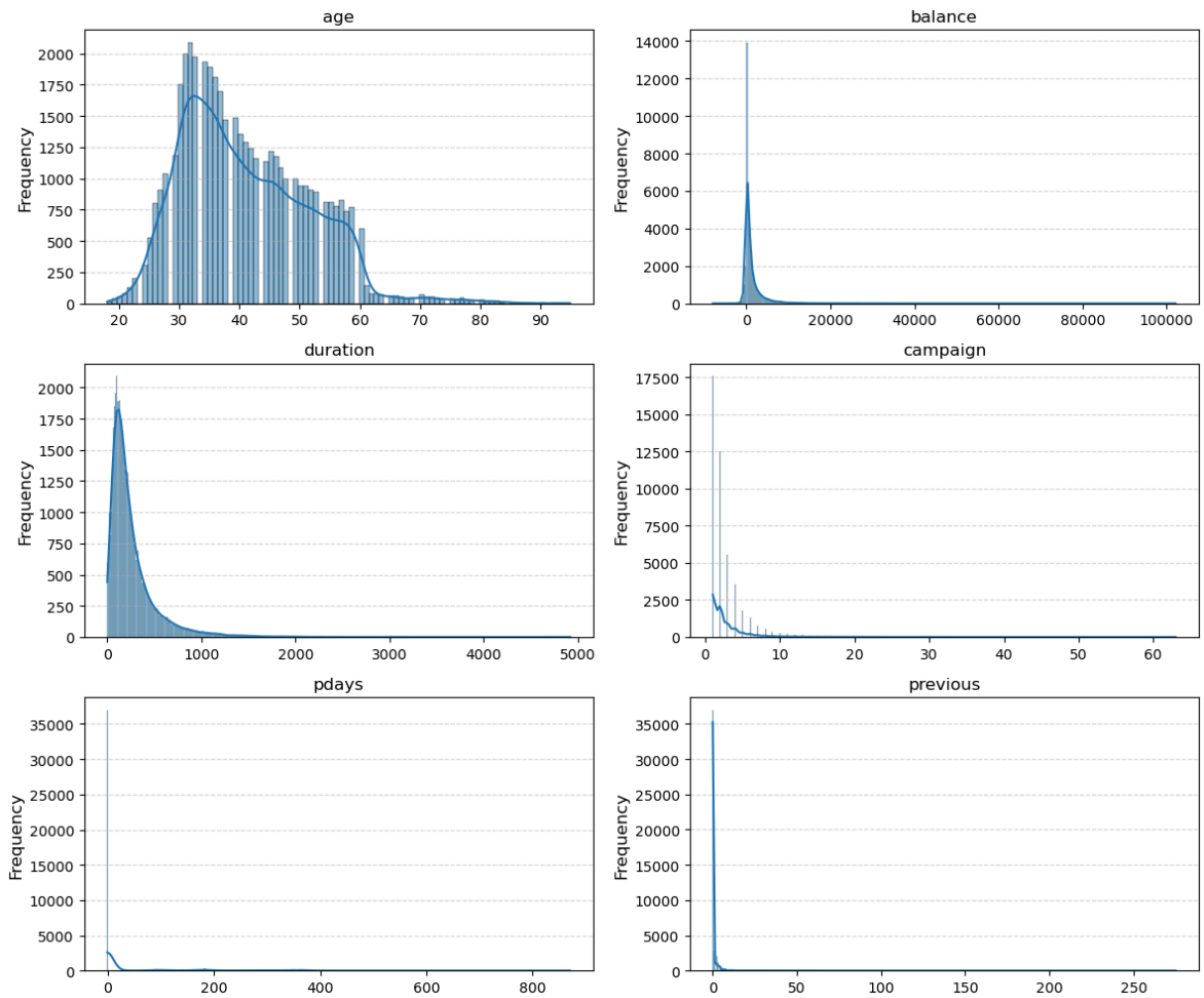
- As variáveis são discretas, mas "duration" apresenta um comportamento contínuo, com assimetria positiva. Adicionalmente, todas elas contam com uma quantidade considerável de outliers.

```
In [19]: fig, axes = plt.subplots(3, 2, figsize=(12, 10))
axes = axes.flatten()

for ax, col in zip(axes, cols_num):
    sns.histplot(
        data=df_eda,
        x=col,
        kde=True,
        edgecolor="black",
        ax=ax
    )
    ax.set_title(col, fontsize=12)
    ax.set_xlabel('')
    ax.set_ylabel("Frequency", fontsize=12)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_num):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```

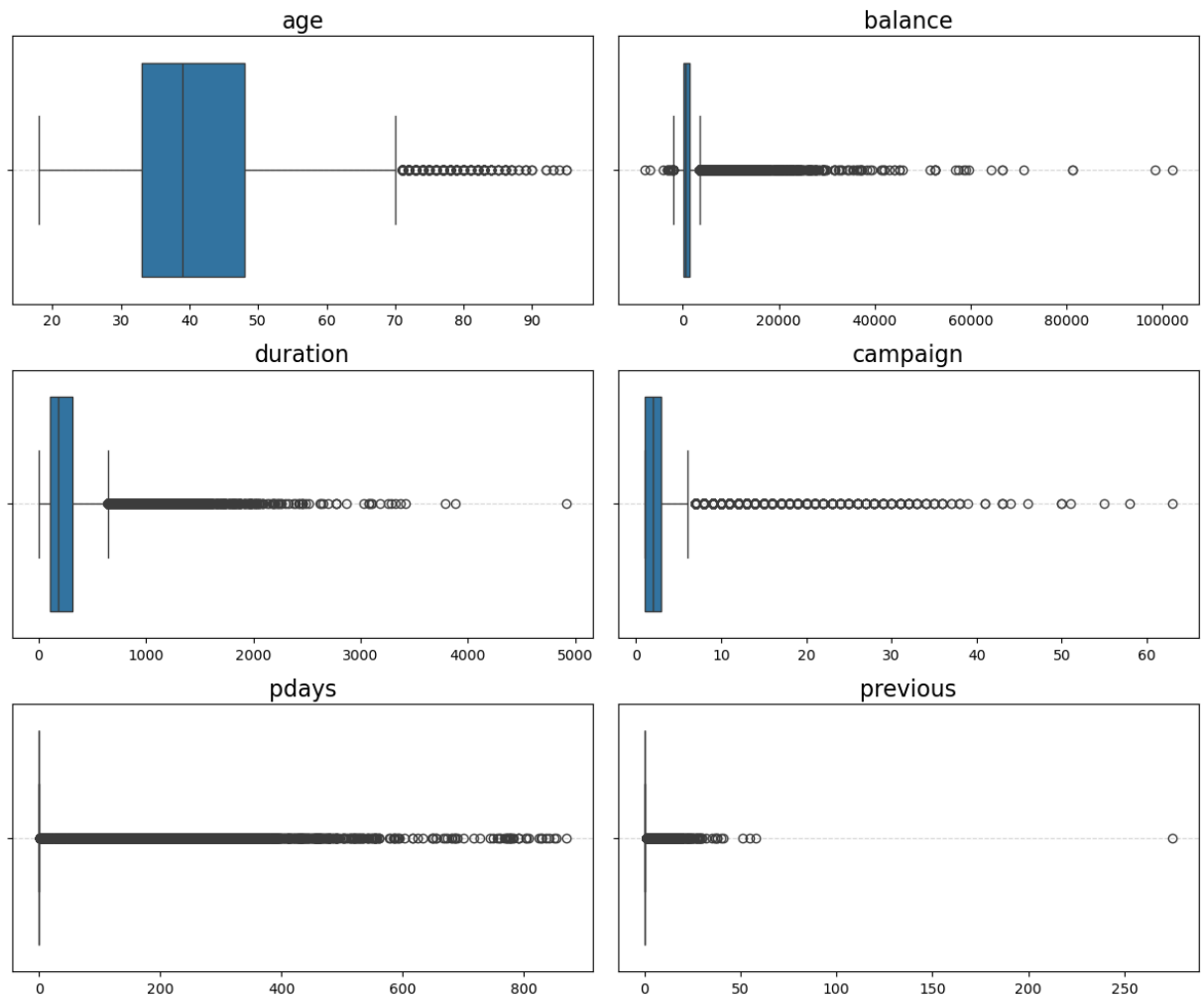


```
In [20]: fig, axes = plt.subplots(3, 2, figsize=(12, 10))
         axes = axes.flatten()

         for ax, col in zip(axes, cols_num):
             sns.boxplot(
                 data=df_eda,
                 x=col,
                 ax=ax
             )
             ax.set_title(col, fontsize=16)
             ax.set_xlabel('')
             ax.grid(axis="y", linestyle="--", alpha=0.5)

         # Caso tenha mais axes do que colunas numéricas
         for ax in axes[len(cols_num):]:
             fig.delaxes(ax)

         plt.tight_layout()
         plt.show()
```



- Distribuição das variáveis binárias

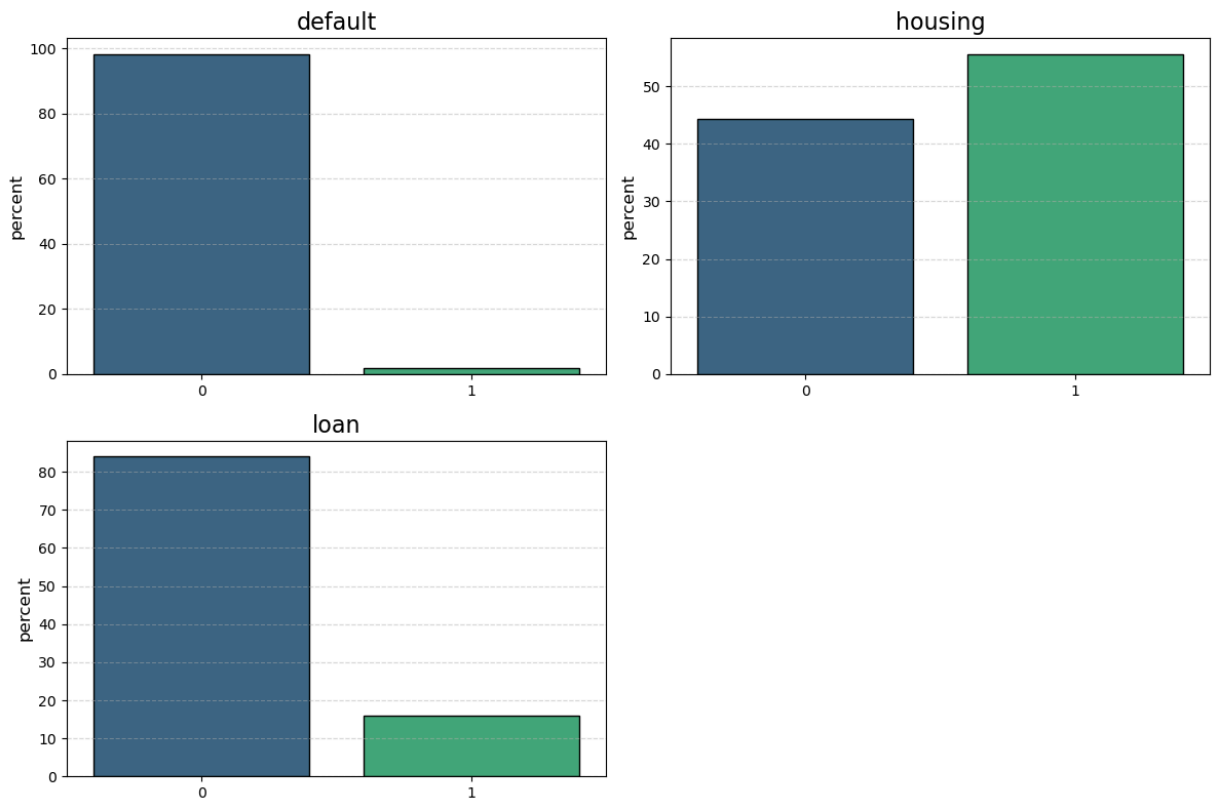
- As variáveis default e loan são extremamente desbalanceadas, isso pode levar a problemas em testes de associação como o V de Cramer.

```
In [21]: fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, col in zip(axes, cols_bin):
    sns.countplot(
        data=df_eda,
        x=col,
        stat="percent",
        edgecolor="black",
        palette="viridis",
        ax=ax
    )
    ax.set_title(col, fontsize=16)
    ax.set_xlabel('')
    ax.set_ylabel('percent', fontsize=12)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_bin):]:
    fig.delaxes(ax)
```

```
plt.tight_layout()
plt.show()
```



- Distribuição das variáveis categóricas

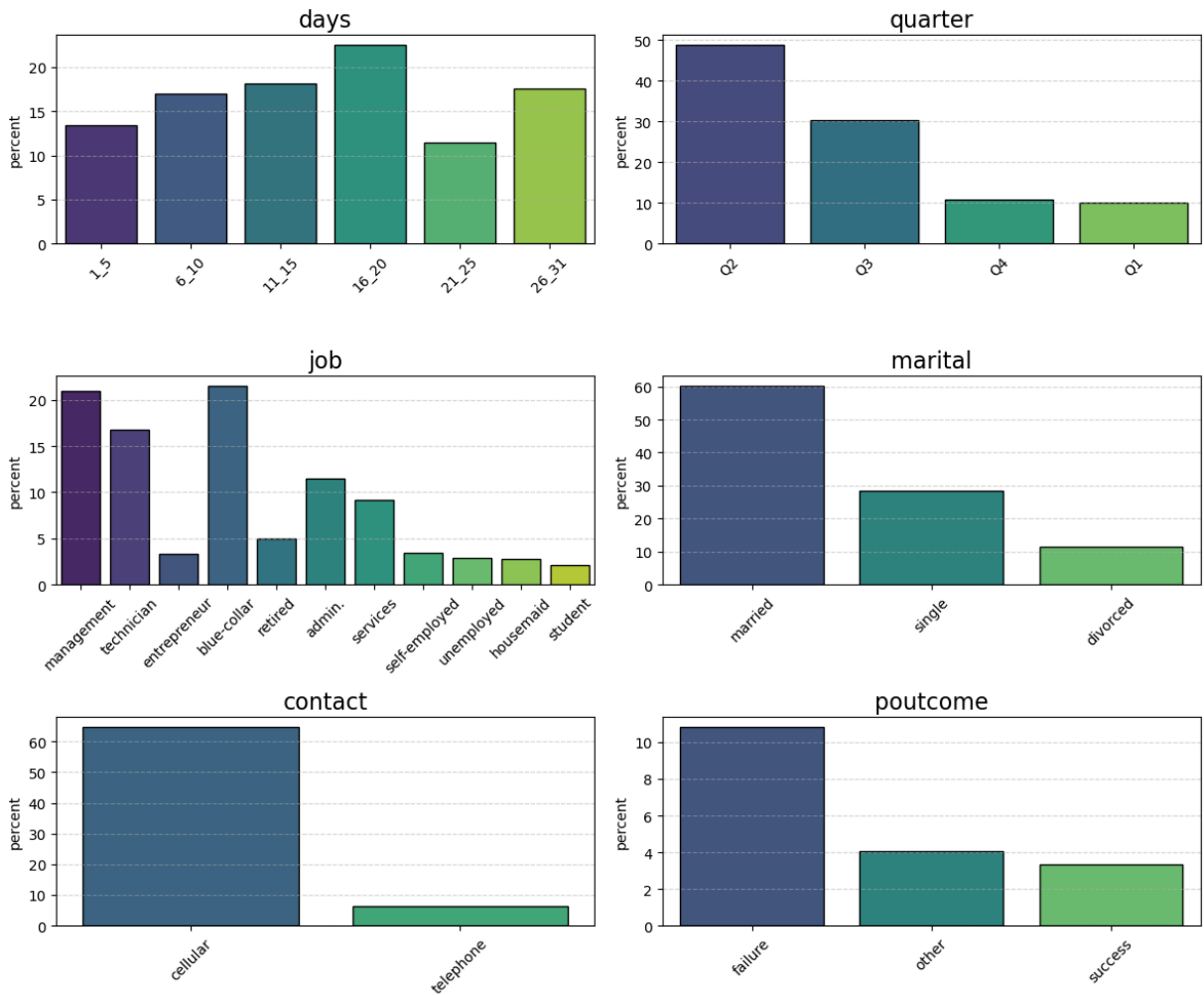
- Os desbalanceamento são mais notáveis nas variáveis contact, poutcome e marital.

```
In [22]: fig, axes = plt.subplots(3, 2, figsize=(12, 10))
         axes = axes.flatten()

         for ax, col in zip(axes, cols_cat):
             sns.countplot(
                 data=df_eda,
                 x=col,
                 stat="percent",
                 edgecolor="black",
                 palette="viridis",
                 ax=ax
             )
             ax.set_title(col, fontsize=16)
             ax.set_xlabel('')
             ax.set_ylabel("percent")
             ax.grid(axis="y", linestyle="--", alpha=0.5)
             ax.tick_params("x", rotation=45)

         # Caso tenha mais axes do que colunas numéricas
         for ax in axes[len(cols_cat):]:
             fig.delaxes(ax)
```

```
plt.tight_layout()
plt.show()
```

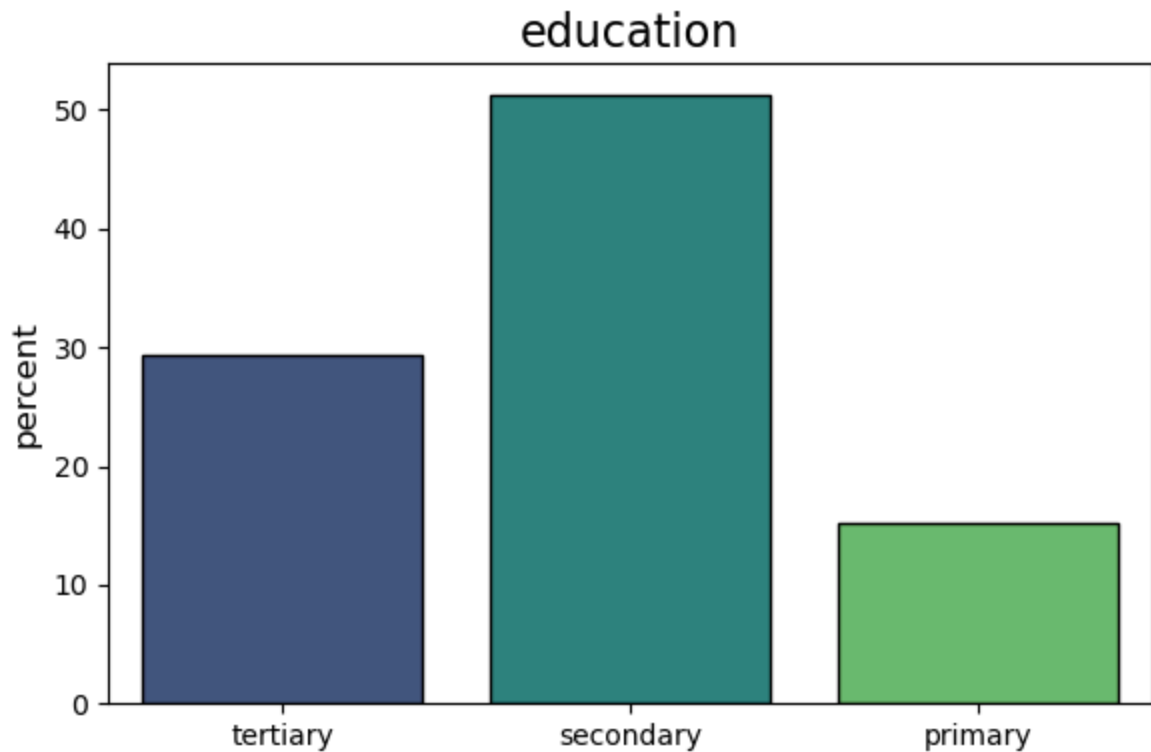


- Distribuição da variável semi-ordinal

- Mais de 50% dos indivíduos têm educação secundária e menos de 20% possuem apenas a primária.

```
In [23]: plt.figure(figsize=(6,4))
sns.countplot(
    data=df_eda,
    x="education",
    stat="percent",
    edgecolor="black",
    palette="viridis"
)

plt.title("education", fontsize=16)
plt.xlabel('', fontsize=12)
plt.ylabel("percent", fontsize=12)
plt.tight_layout()
plt.show();
```



- Relação da variável target com as demais numéricas

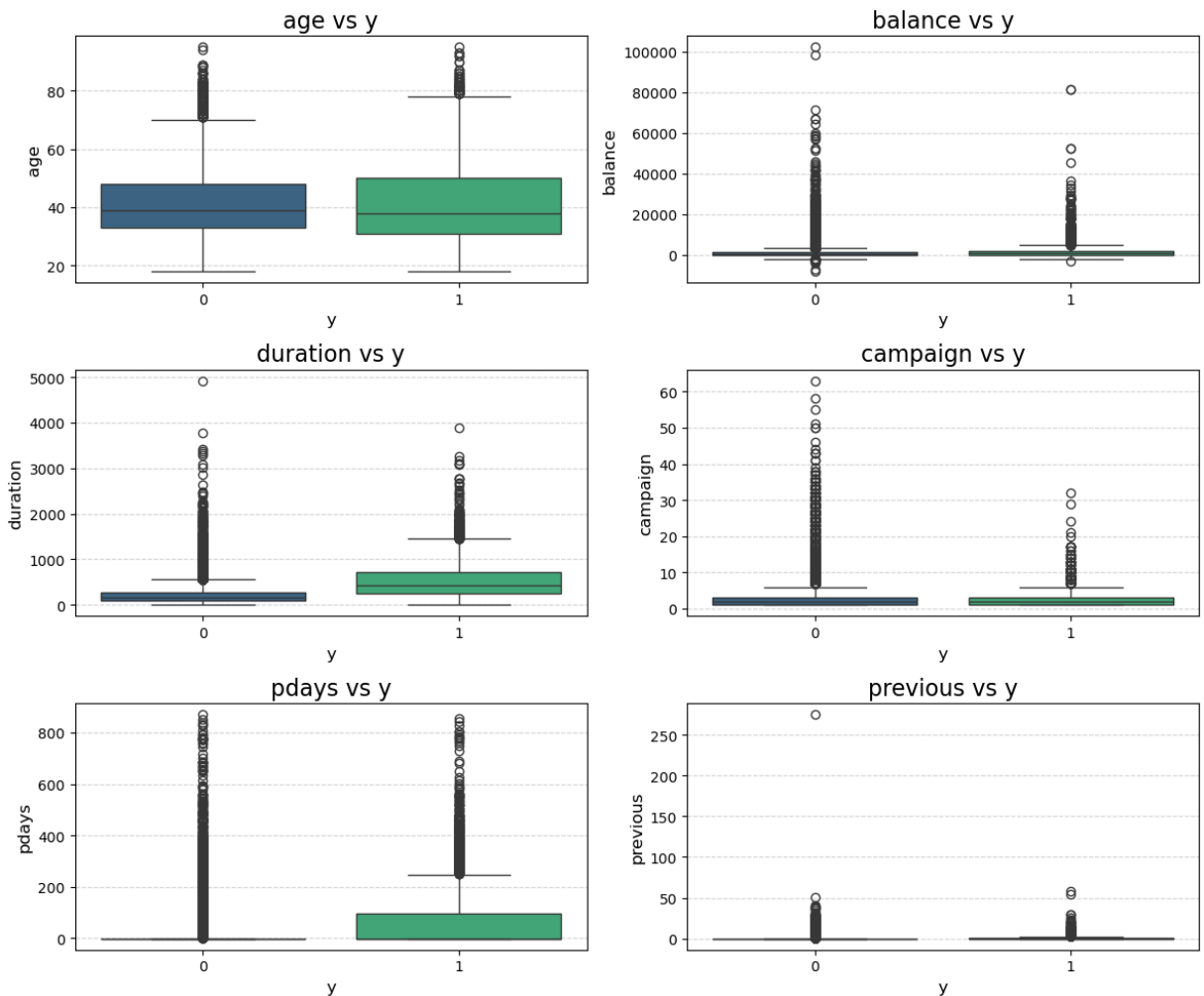
- A mediana entre indivíduos que foram convertidos e não convertidos é equiparável. Interessantemente, os indivíduos que foram convertidos são aqueles com maior tempo de duração de chamada (leakage).

```
In [24]: fig, axes = plt.subplots(3, 2, figsize=(12, 10))
axes = axes.flatten()

for ax, col in zip(axes, cols_num):
    sns.boxplot(
        data=df_eda,
        y=col,
        x="y",
        ax=ax,
        palette="viridis"
    )
    ax.set_title(f"{col} vs {target}", fontsize=16)
    ax.set_xlabel("y", fontsize=12)
    ax.set_ylabel(col, fontsize=12)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_num):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```

- Relação da variável target com as variáveis binárias

- Insights (na média, não determinístico): . Indivíduos inadimplentes estão mais associados a maiores taxas de conversão; . Indivíduos sem empréstimo habitacional estão mais associados a maiores taxas de conversão; . Indivíduos sem empréstimo estão mais associados a maiores taxas de conversão.

```
In [25]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

for ax, col in zip(axes, cols_bin):

    # Tabela de proporções condicionais
    prop = (
        df_eda
        .groupby(col)[target]
        .value_counts(normalize=True)
        .rename("proportion")
        .reset_index()
    )

    sns.barplot(
        data=prop,
        x=col,
        y="proportion",
        hue=target,
```

```

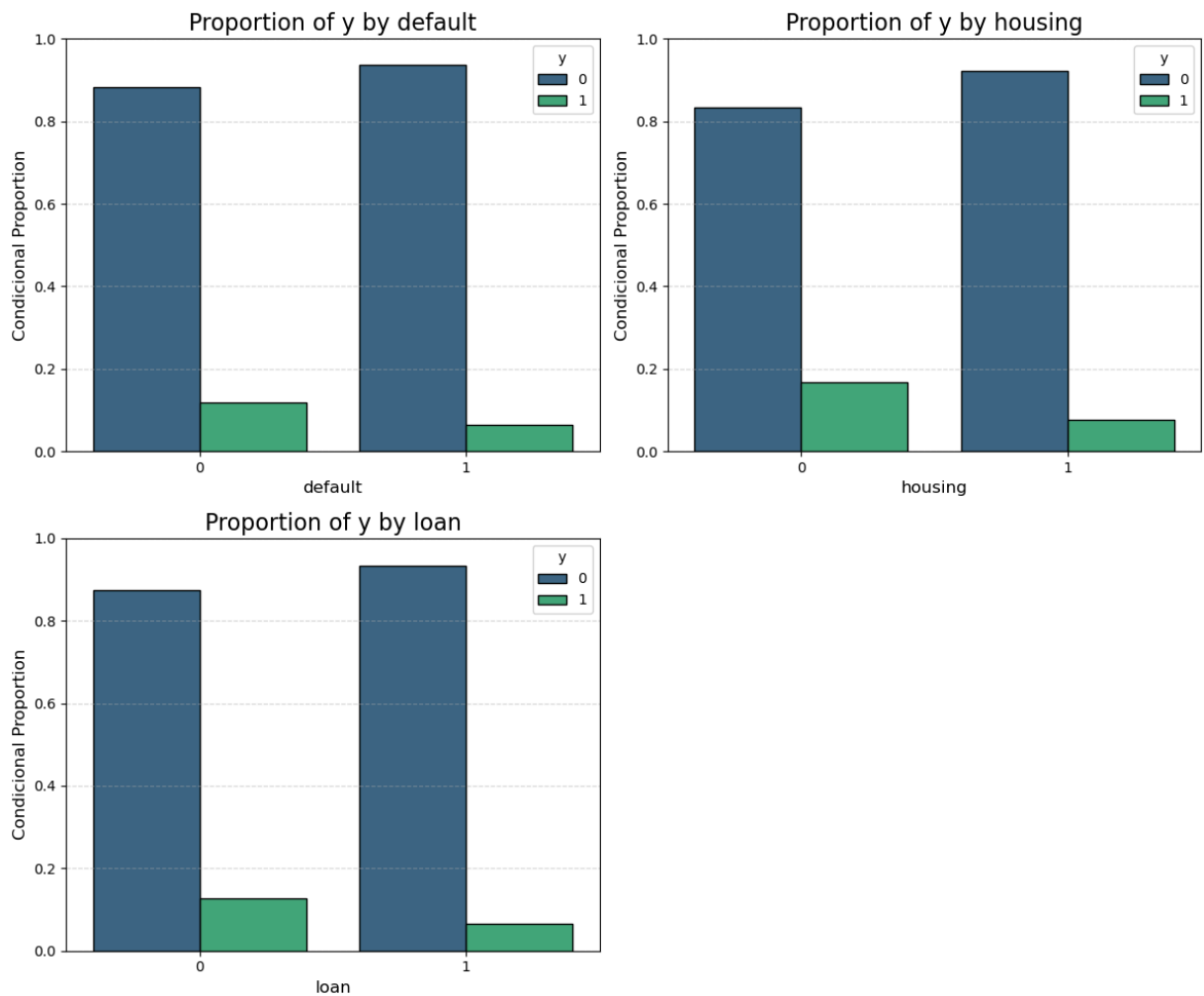
    palette="viridis",
    ax=ax,
    edgecolor="black"
)

ax.set_title(f"Proportion of {target} by {col}", fontsize=16)
ax.set_xlabel(col, fontsize=12)
ax.set_ylabel("Condicional Proportion", fontsize=12)
ax.set_ylim(0, 1)
ax.grid(axis="y", linestyle="--", alpha=0.5)
ax.tick_params("x")
ax.tick_params("y")

# Remover axes excedentes
for ax in axes[len(cols_bin):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()

```



- Relação da variável target com as variáveis categóricas

- Insights (na média, não determinístico): . Dias 1-5 estão associados a maiores taxas de conversão; . Primeiro trimestre está associado a maiores taxas de conversão; . Estudantes estão mais associados a maiores taxas de conversão; . Solteiros estão mais

associados a maiores taxas de conversão; . O contato feito por celular está mais associado (ligeiramente) a maiores taxas de conversão; . Campanhas passadas que obtiveram sucesso estão muito mais associadas a maiores taxas de conversão.

```
In [26]: fig, axes = plt.subplots(3, 2, figsize=(15, 12))
axes = axes.flatten()

for ax, col in zip(axes, cols_cat):

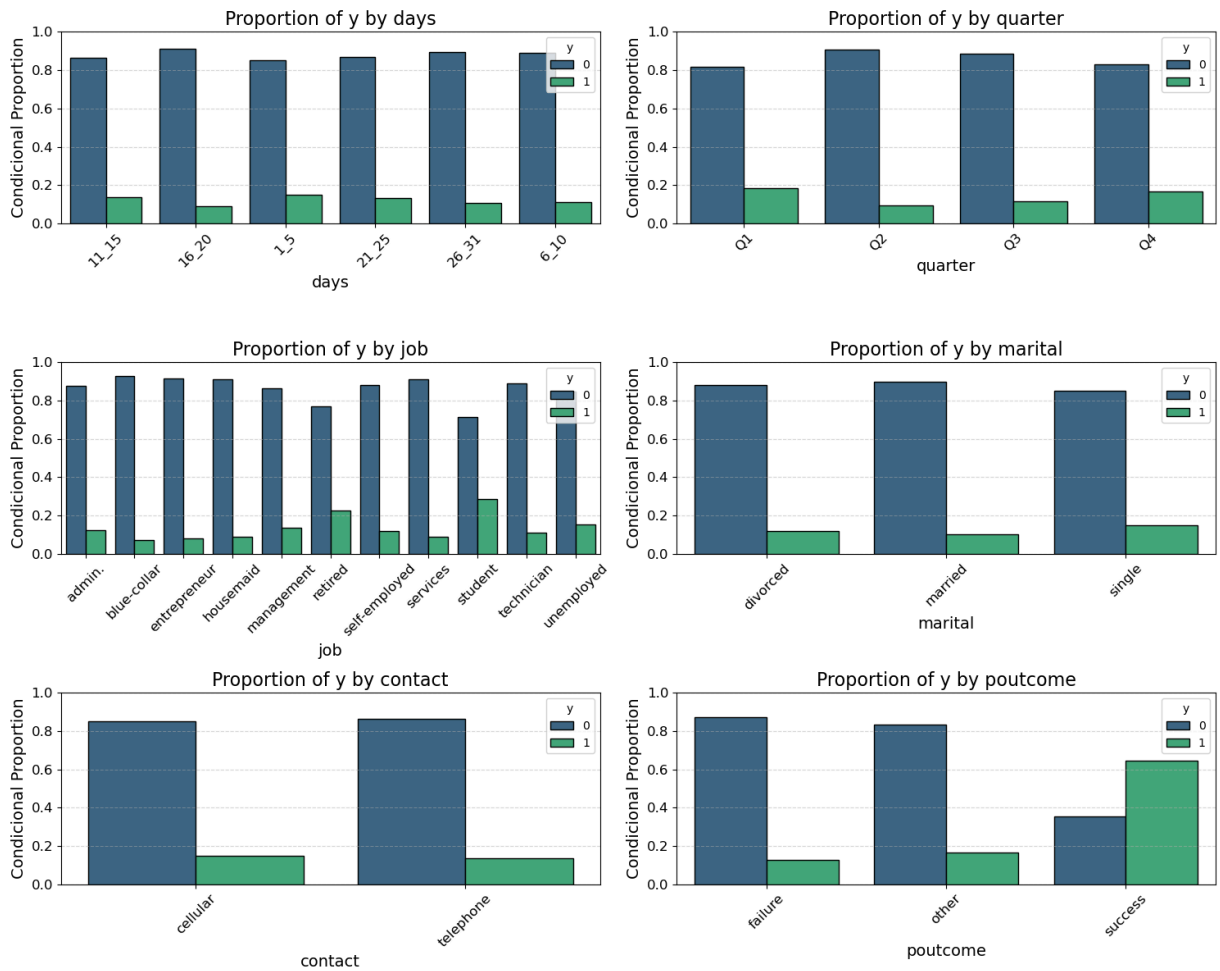
    # Tabela de proporções condicionais
    prop = (
        df_eda
        .groupby(col)[target]
        .value_counts(normalize=True)
        .rename("proportion")
        .reset_index()
    )

    sns.barplot(
        data=prop,
        x=col,
        y="proportion",
        hue=target,
        palette="viridis",
        ax=ax,
        edgecolor="black"
    )

    ax.set_title(f"Proportion of {target} by {col}", fontsize=16)
    ax.set_xlabel(col, fontsize=14)
    ax.set_ylabel("Conditional Proportion", fontsize=14)
    ax.set_ylim(0, 1)
    ax.grid(axis="y", linestyle="--", alpha=0.5)
    ax.tick_params("x", rotation=45, labels=12)
    ax.tick_params("y", labels=12)

# Remover axes excedentes
for ax in axes[len(cols_cat):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```



- Relação da variável target com a variável semi-ordinária

- Indivíduos com escolaridade terciária, em média, estão mais associados a maiores taxas de conversão.

```
In [27]: fig, axes = plt.subplots(1, 1, figsize=(6, 4))

if not isinstance(axes, np.ndarray):
    axes = [axes]

for ax, col in zip(axes, cols_semiord):

    # Tabela de proporções condicionais
    prop = (
        df_eda
        .groupby(col)[target]
        .value_counts(normalize=True)
        .rename("proportion")
        .reset_index()
    )

    sns.barplot(
        data=prop,
        x=col,
        y="proportion",
        hue=target,
        palette="viridis",
```

```

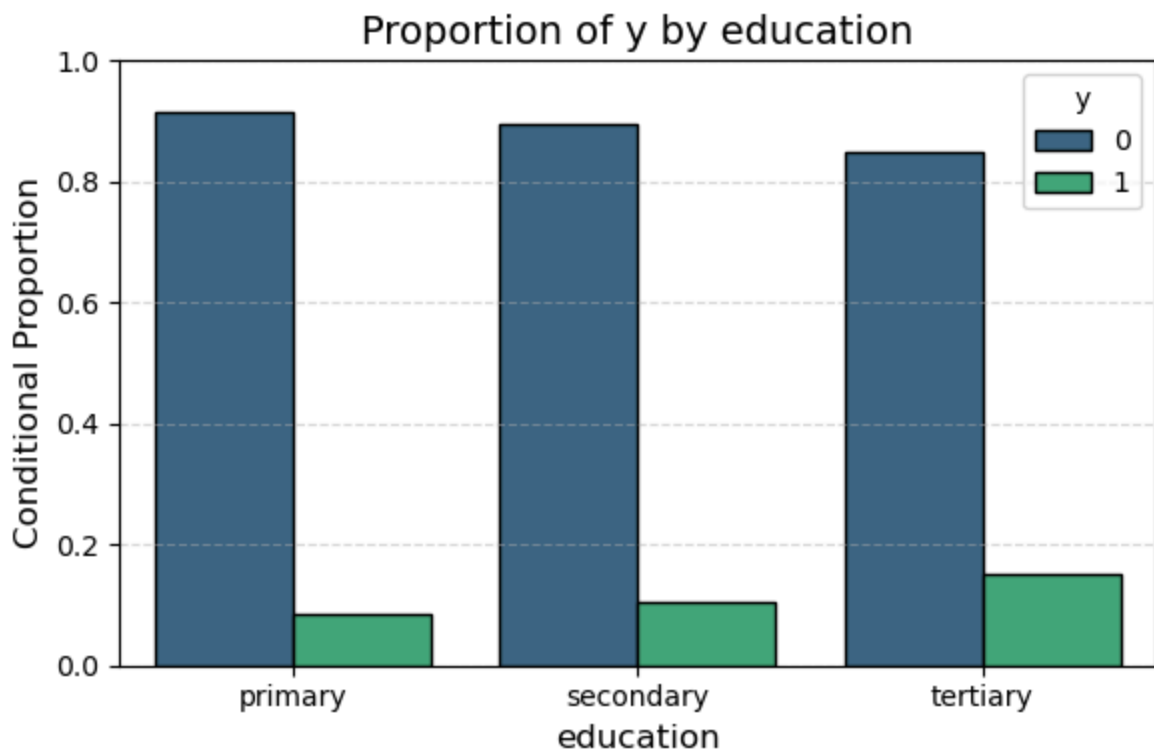
        ax=ax,
        edgecolor="black"
    )

    ax.set_title(f"Proportion of {target} by {col}", fontsize=14)
    ax.set_xlabel(col, fontsize=12)
    ax.set_ylabel("Conditional Proportion", fontsize=12)
    ax.set_ylim(0, 1)
    ax.grid(axis="y", linestyle="--", alpha=0.5)
    ax.tick_params("x", labels=10)

if len(axes) > len(cols_semiord):
    for i in range(len(cols_semiord), len(axes)):
        fig.delaxes(axes[i])

plt.tight_layout()
plt.show()

```



- Correlação

- A correlação linear entre as variáveis varia de fraca a moderada.

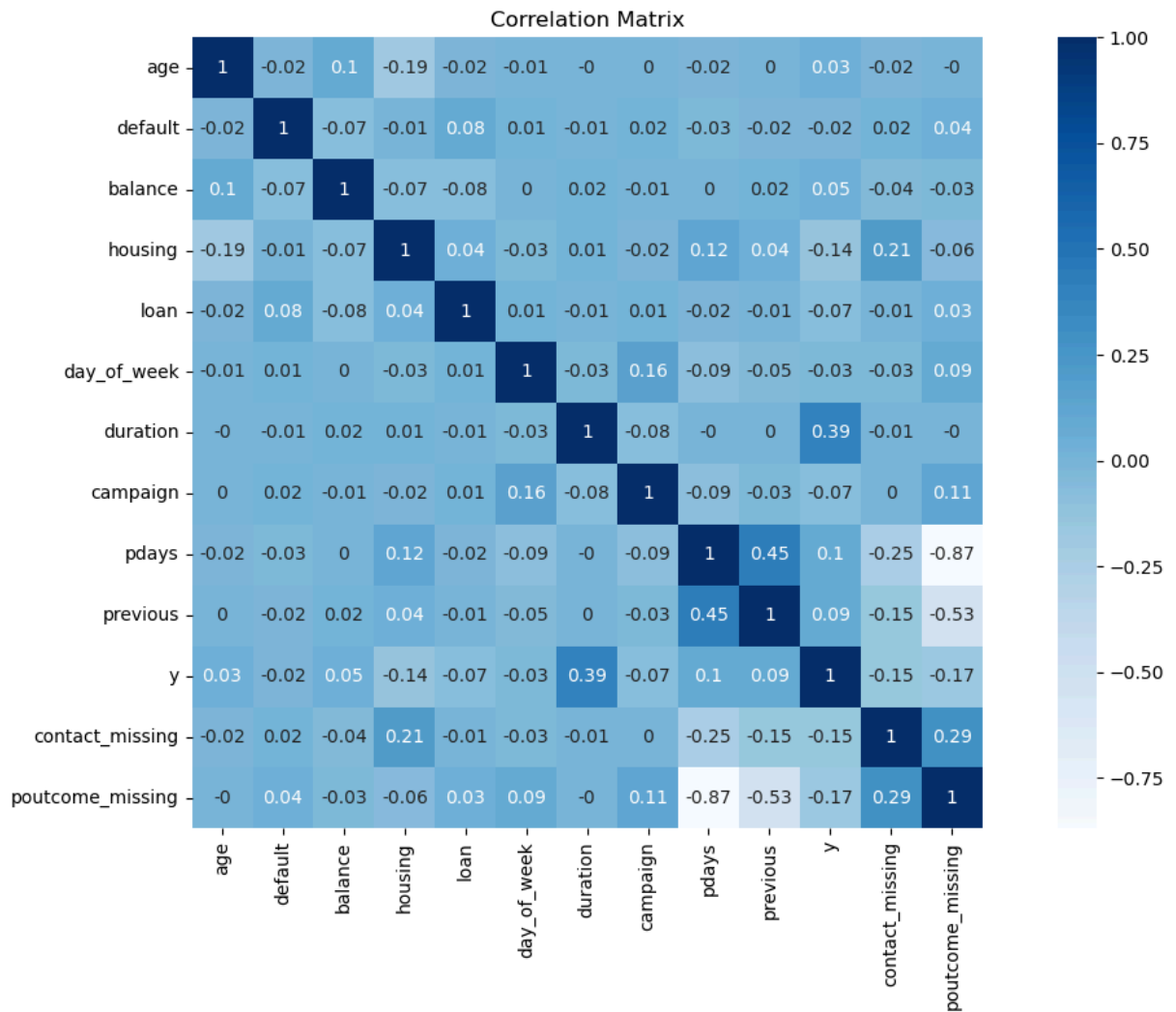
```

In [28]: plt.figure(figsize = (12,8))
plt.title("Correlation Matrix")

corr = df_eda.corr(numeric_only = True)
sns.heatmap(corr.round(2), vmax = 1, square = True, annot = True, cmap = "Blues")

plt.tight_layout()
plt.show()

```



- Associação

Valor do V de Cramér Interpretação da Associação Comentário de Negócio -----

----- 0.00 a 0.10 Desprezível / Nula A variável não ajuda a distinguir as classes. Ruído. 0.10 a 0.30 Fraca Existe um sinal leve, mas sozinha a variável não decide nada. Útil em conjunto com outras. 0.30 a 0.50 Moderada Associação Importante. Esta variável já tem poder preditivo relevante. > 0.50 Forte Determinante. A variável quase "define" o resultado. (Raro em dados sociais/humanos).- Destaca-se a relação entre job e education (0.55). Os indivíduos com maior nível de educação, em média, são aqueles com os "melhores" empregos em termos de remuneração. Entretanto, apesar de educação ser tratado como variável semi-ordinal, job é puramente categórica. - O financiamento de imóvel aparenta ter uma dependência moderada com o trimestre do ano. A sazonalidade pode estar desempenhando algum papel (início de ano mais aquecido por campanhas e fim de ano com baixa na procura, favorecendo negociação) e cenário econômico. - O financiamento de imóvel também aparenta ter uma dependência moderada com o resultado da campanha de marketing passada. Campanhas bem-sucedidas geram leads e visibilidade, que se convertem em vendas e, consequentemente, em aumento do volume de financiamentos. - Apesar do V de Cramer trazer insights que fazem sentido, não se pode confiar muito devido a natureza problemática do dataset.

```
In [29]: def cramers_v_corrected(confusion_matrix):
chi2 = stats.chi2_contingency(confusion_matrix)[0]
n = confusion_matrix.sum().sum()
phi2 = chi2 / n
r, k = confusion_matrix.shape

phi2corr = max(0, phi2 - ((k - 1)*(r - 1))/(n - 1))
rcorr = r - (r - 1)**2 / (n - 1)
```

```

kcorr = k - (k - 1)**2 / (n - 1)

return np.sqrt(phi2corr / min((kcorr - 1), (rcorr - 1)))

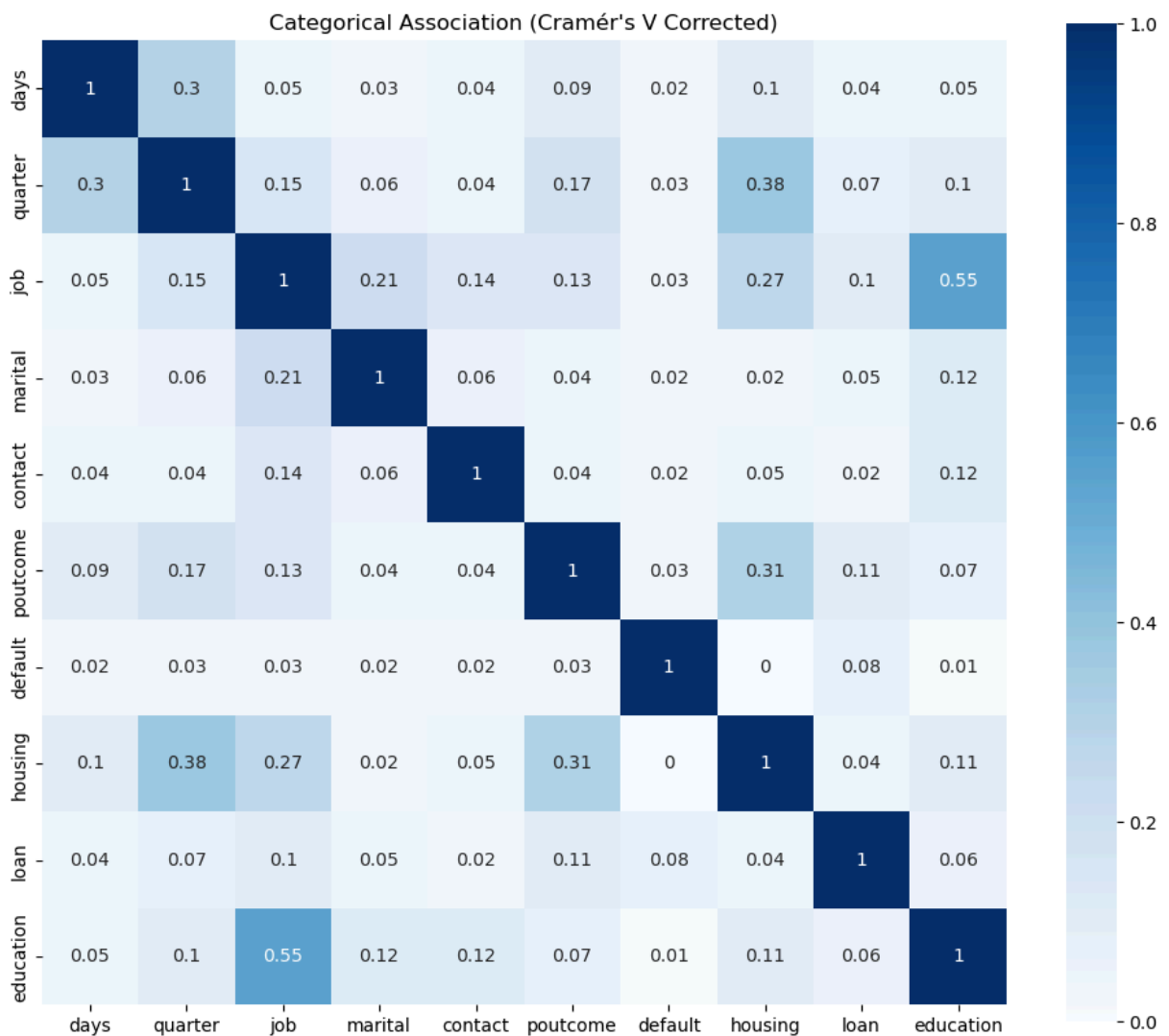
cols_v = cols_cat + cols_bin + cols_semiord

mat = pd.DataFrame(index=cols_v, columns=cols_v)

for c1 in cols_v:
    for c2 in cols_v:
        tab = pd.crosstab(df_eda[c1], df_eda[c2])
        mat.loc[c1, c2] = cramers_v_corrected(tab.values)

plt.figure(figsize=(12,10))
sns.heatmap(mat.astype(float).round(2), annot=True, square=True, cmap='Blues', vmin
plt.title("Categorical Association (Cramér's V Corrected)")
plt.show()

```



- Data leakage

Condição Interpretação (heurística) -----
----- z_score > 5 Leakage muito provável 3 < z_score ≤ 5 Leakage conceitual/proxy suspeito z_score ≤ 3
Informação plausível -----

----- Segundo o mutual information, duration, contact e poutcome são: . Diretamente derivadas do evento-alvo, ou . Disponível apenas após o desfecho, ou . Codifica uma proxy temporal do target. - Decisão: Remover.

```
In [30]: leakage_features = []

def leakage_mi_permutation_test(
    df,
    feature,
    target="y",
    n_permutations=20,
    random_state=42
):
    # Checagem mínima
    if feature not in df.columns or target not in df.columns:
        raise ValueError("Feature ou target inexistente.")

    df_local = df[[feature, target]].dropna()

    X = df_local[[feature]]
    y = df_local[target]

    # MI real
    mi_real = mutual_info_classif(
        X, y,
        discrete_features="auto",
        random_state=random_state
    )[0]

    # MI sob permutação do target
    rng = np.random.RandomState(random_state)
    mi_perm = []

    for _ in range(n_permutations):
        y_perm = rng.permutation(y)
        mi_p = mutual_info_classif(
            X, y_perm,
            discrete_features="auto",
            random_state=random_state
        )[0]
        mi_perm.append(mi_p)

    mi_perm = np.array(mi_perm)

    delta = mi_real - mi_perm.mean()
    z_score = delta / (mi_perm.std() + 1e-8)

    return {
        "feature": feature,
        "mi_real": mi_real,
        "mi_perm_mean": mi_perm.mean(),
        "delta_mi": delta,
        "z_score": z_score
    }
```



```
In [31]: result = leakage_mi_permutation_test(df_eda, feature="duration") # feature = variáv

if result["z_score"] > 5:
    leakage_features.append("duration")

print(f"Feature causing leakage: {leakage_features}")
```

Feature causing leakage: ['duration']

```
In [32]: result = leakage_mi_permutation_test(df_eda, feature="contact_missing") # feature =

if result["z_score"] > 5:
    leakage_features.append("contact_missing")

print(f"Feature causing leakage: {leakage_features}")
```

Feature causing leakage: ['duration', 'contact_missing']

```
In [33]: result = leakage_mi_permutation_test(df_eda, feature="poutcome_missing") # feature

if result["z_score"] > 5:
    leakage_features.append("poutcome_missing")

print(f"Feature causing leakage: {leakage_features}")
```

Feature causing leakage: ['duration', 'contact_missing', 'poutcome_missing']

Pré-processamento

- A etapa de pré-processamento teve como objetivo preparar o conjunto de dados para a modelagem preditiva, garantindo consistência, validade estatística e aderência a um cenário real de aplicação. As decisões adotadas foram diretamente orientadas pelos achados da análise exploratória e pelas características estruturais do dataset.

- Tratamento da variável alvo: . A variável target (y) é binária e apresenta desbalanceamento significativo entre as classes. Esse aspecto foi preservado no conjunto de dados original, sendo tratado posteriormente no pipeline de modelagem por meio de técnicas específicas, evitando alterações artificiais na distribuição durante a fase exploratória.
- Exclusão de variáveis com risco de data leakage: . As variáveis duration, poutcome e contact foram excluídas por violarem pressupostos fundamentais de temporalidade e independência causal. duration constitui uma variável pós-evento diretamente influenciada pelo desfecho, caracterizando vazamento explícito. Já poutcome e contact codificam decisões operacionais históricas da instituição, introduzindo vazamento conceitual e viés de política. Testes empíricos (AUC univariado, estabilidade temporal, permutation importance e análise SHAP) corroboram essa decisão, evidenciando capacidade preditiva desproporcional e instabilidade fora da amostra.
- Tratamento de valores ausentes: . Na primeira etapa, detectou-se valores faltantes nas variáveis como job e education. Para essas variáveis, adotou-se imputação pelo valor mais frequente, uma escolha consistente com a natureza discreta das categorias e que evita a introdução de vieses artificiais; . As variáveis numéricas não apresentaram valores ausentes relevantes, não exigindo intervenções adicionais nessa etapa.
- Codificação de variáveis categóricas: . As variáveis categóricas nominais foram transformadas por meio de one-hot encoding, permitindo sua utilização por modelos que operam exclusivamente com entradas numéricas. A variável education, embora possua uma ordenação conceitual, foi tratada como categórica, uma vez que não se assume relação linear ou monotônica entre seus níveis no contexto do problema analisado; . Essa abordagem preserva a flexibilidade do modelo para capturar relações não lineares entre níveis educacionais e a probabilidade de subscrição.
- Padronização das variáveis numéricas: . As variáveis numéricas foram padronizadas utilizando standard scaling, garantindo média zero e desvio padrão unitário. Embora modelos baseados em árvores não sejam sensíveis à escala, a padronização foi aplicada de forma consistente a todo o pipeline, assegurando comparabilidade entre diferentes algoritmos, especialmente aqueles baseados em otimização por gradiente, como regressão logística.
- Organização do pipeline de pré-processamento: . Todas as transformações foram organizadas em um pipeline unificado por meio de um ColumnTransformer, assegurando que as mesmas etapas fossem aplicadas de forma consistente aos conjuntos de treino e teste. Essa abordagem reduz riscos de vazamento de informação, facilita a reprodução dos experimentos e garante alinhamento entre pré-processamento e modelagem.

- Dataset de pré-processamento

```
In [34]: df_prep = df_eda.copy()
```

- Tratamento de valores duplicados e valores faltantes

- Os valores duplicados foram tratados antes, na seção EDA. As variáveis contact e poutcome que continham valores faltantes eram suspeitas de ausência não aleatória e que causavam vazamento de dados, foi confirmada via mutual information. Desse modo, duration, contact e poutcome devem ser removidas antes da modelagem. - As variáveis que ainda continham valores faltantes, mas não eram suspeitas serão preenchidas com sua mediana (robusta a outliers) ou valor mais frequente, a depender do seu tipo.

```
In [35]: imputer_semior = SimpleImputer(strategy="most_frequent") # education
imputer_nominal = SimpleImputer(strategy="most_frequent") # job
```

- Remoção de features

- Remover features que serviram de base para transformações e as causadoras de data leakage.

```
In [36]: df_prep = df_prep.drop(columns = ["day_of_week", "month", "duration",
                                           "contact", "poutcome",
                                           "contact_missing", "poutcome_missing"]).reset_index()
```

- Pré-processador

```
In [37]: cols_num = ["age", "balance", "campaign", "pdays", "previous"]

cols_bin = ["default", "housing", "loan"]

cols_cat = ["days", "quarter", "job", "marital"]

cols_semiord = ["education"]

target = "y"
```

```
In [38]: encoder_nominal = OneHotEncoder(drop="first", handle_unknown="ignore")
```

```
In [39]: preprocessor = ColumnTransformer(
    transformers=[
        # Numéricas – padronizadas para TODOS os modelos
        ("num", StandardScaler(), cols_num),

        # Binárias – já numéricas, sem imputação
        ("bin", "passthrough", cols_bin),

        # Semi-ordinal
        ("semior", Pipeline(steps=[
            ("imputer", imputer_semior),
            ("onehot", encoder_nominal)
        ]), cols_semiord),

        # Categóricas nominais
```

```

        ("cat", Pipeline(steps=[
            ("imputer", imputer_nominal),
            ("onehot", encoder_nominal)
        ]), cols_cat),
    ],
    remainder="drop"
)

```

- Separar X e Y

```

In [40]: X = df_prep.drop(columns = ["y"])
        y = df_prep["y"]

```

- Dividir em treino/teste (antes de fit)

```

In [41]: # Stratify em y é essencial para um dataset desbalanceado
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

```

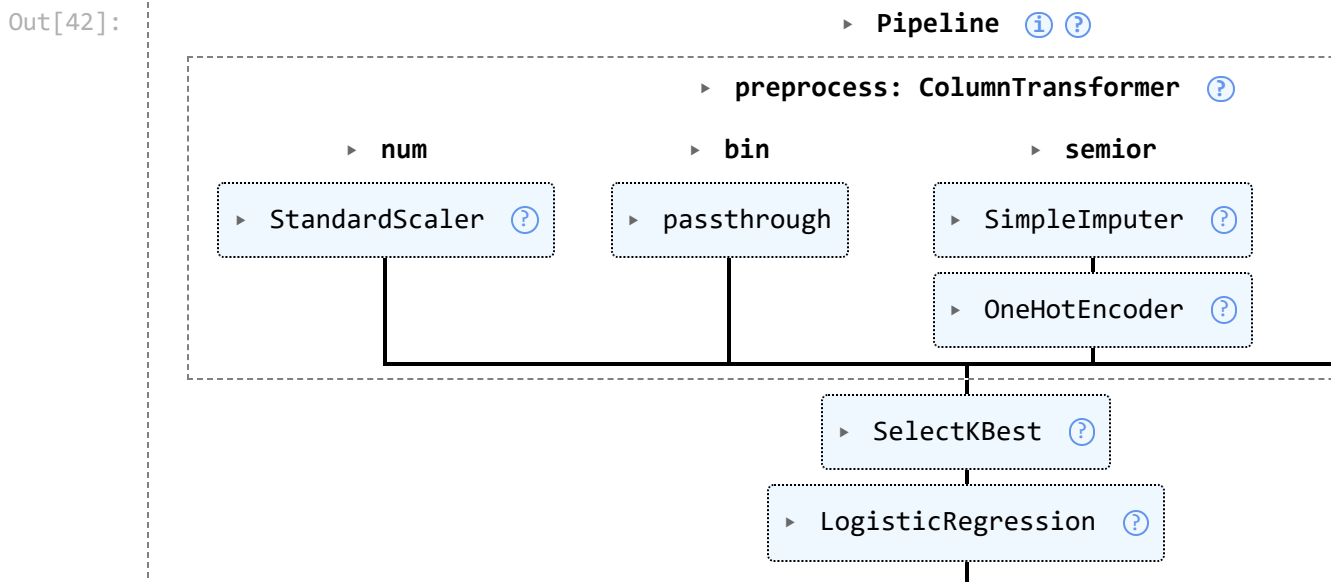
- Seleção de Features

```

In [42]: pipeline = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("feature_selection", SelectKBest(score_func=mutual_info_classif, k="all")),
        ("model", LogisticRegression(max_iter=1000, class_weight="balanced"))
    ])

        pipeline.fit(X_train, y_train)

```



Treinamento e Avaliação de Modelos Base

- Sintetizando a opção de modelos promissores (ordem de prioridade): . Conversion Rate @ K% (primária): LGBM e XGBoost; . Lift @ K% (primária): LGBM e XGBoost; . Average Precision (secundária): LGBM e XGBoost; . ROC-AUC (secundária):

- Criar avaliador universal

```
In [43]: # =====
# Avaliador
# =====

def evaluate_model(
    name,
    pipeline,
    X_test,
    y_test,
    ks=(0.01, 0.05),
    plot_confusion=True
):
    """
    Avaliador universal para Bank Marketing (subscrição de depósito).

    Objetivo REAL:
    - Priorizar clientes com maior probabilidade de conversão
    - Avaliação baseada em ranking (não em threshold fixo)

    Notas importantes:
    - Threshold 0.5 é APENAS diagnóstico
    - Comparação de modelos deve usar métricas threshold-independentes
      e métricas de ranking (Precision@K / Lift@K)
    """

    print(f"\n{'='*65}")
    print(f"MODEL: {name}")
    print(f"{'='*65}")

    # -----
    # 1. Probabilidades previstas
    # -----
    y_proba = pipeline.predict_proba(X_test)[: , 1]

    # -----
    # 2. Métricas globais (threshold-independentes)
    # -----
    roc_auc = roc_auc_score(y_test, y_proba)
    ap = average_precision_score(y_test, y_proba)

    print("\nGLOBAL METRICS:")
    print(f"  ROC-AUC:          {roc_auc:.4f}")
    print(f"  Average Precision: {ap:.6f}")

    # -----
    # 3. Métricas de ranking (marketing direto)
    # -----
    base_rate = y_test.mean()

    print("\nRANKING METRICS (CAMPAIGN TRIAGE):")
    print(f"  Base conversion rate: {base_rate:.4f}")
```

```

n = len(y_proba)

for k in ks:
    cutoff = int(n * k)
    idx = np.argsort(y_proba)[:k][::-1][:cutoff]

    # Taxa de conversão no Top-K (Precision@K)
    precision_k = y_test.iloc[idx].mean()

    # Lift relativo ao contato aleatório
    lift_k = precision_k / base_rate if base_rate > 0 else np.nan

    print(f"\n Top {int(k * 100)}% clients:")
    print(f"      Conversion Rate (Precision@K): {precision_k:.4f}")
    print(f"      Lift@K: {lift_k:.2f}")

# -----
# 4. Diagnóstico com threshold fixo (0.50)
# -----
y_pred = (y_proba >= 0.5).astype(int)

print("\nDIAGNOSTIC REPORT (Threshold = 0.50):")
print(classification_report(y_test, y_pred, digits=4))

if plot_confusion:
    fig, ax = plt.subplots(figsize=(5, 4))
    ConfusionMatrixDisplay.from_predictions(
        y_test,
        y_pred,
        ax=ax,
        cmap="Blues",
        values_format="d"
    )
    plt.title(f"Confusion Matrix - {name}")
    plt.tight_layout()
    plt.show()

# -----
# 5. Retorno estruturado
# -----
return {
    "Model": name,
    "ROC AUC": roc_auc,
    "Average Precision": ap,
    "Base Rate": base_rate,
    "Y Proba": y_proba
}

```

- Pipeline final para cada modelo

- Por heurística: . Classe moderadamente desbalanceada (1:10) ---> class_weight ou scale_pos_weight; . Classe raríssima (1:1000+) ---> APENAS scale_pos_weight.

```

In [44]: y_train_num = y_train.astype(int)
         y_test_num  = y_test.astype(int)

```

```
scale_pos_weight = (y_train_num == 0).sum() / (y_train_num == 1).sum()
```

```
In [45]: # Dummy - Mais frequente (lower bound)
dummy_pipe = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", DummyClassifier(strategy="most_frequent", random_state=42))
])

dummy_pipe.fit(X_train, y_train_num)
evaluate_model("Baseline - Most Frequent", dummy_pipe, X_test, y_test_num)

dummy_strat = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", DummyClassifier(strategy="stratified", random_state=42))
])

# Dummy - Stratified (baseline probabilístico)
dummy_strat.fit(X_train, y_train_num)
evaluate_model("Baseline - Stratified", dummy_strat, X_test, y_test_num)

# Regressão Logística
logreg_pipe = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=42
    ))
])

logreg_pipe.fit(X_train, y_train_num)
evaluate_model("Logistic Regression", logreg_pipe, X_test, y_test_num)

# Random Forest
rf_pipe = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", RandomForestClassifier(
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ))
])

rf_pipe.fit(X_train, y_train_num)
evaluate_model("Random Forest", rf_pipe, X_test, y_test_num)

# XGBoost
xgb_pipe = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", XGBClassifier(
        scale_pos_weight=scale_pos_weight,
        random_state=42,
        verbosity=0,
        n_jobs=-1
    ))
])
```

```

])

xgb_pipe.fit(X_train, y_train_num)
evaluate_model("XGBoost", xgb_pipe, X_test, y_test_num)

# LGBM
lgbm_pipe = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", LGBMClassifier(
        scale_pos_weight=scale_pos_weight,
        random_state=42,
        n_jobs=-1
    ))
])

lgbm_pipe.fit(X_train, y_train_num)
evaluate_model("LightGBM", lgbm_pipe, X_test, y_test_num)

```

```

=====
MODEL: Baseline - Most Frequent
=====

```

GLOBAL METRICS:

```

ROC-AUC:          0.5000
Average Precision: 0.116997

```

RANKING METRICS (CAMPAIGN TRIAGE):

```

Base conversion rate: 0.1170

```

Top 1% clients:

```

Conversion Rate (Precision@K): 0.0889
Lift@K:          0.76

```

Top 5% clients:

```

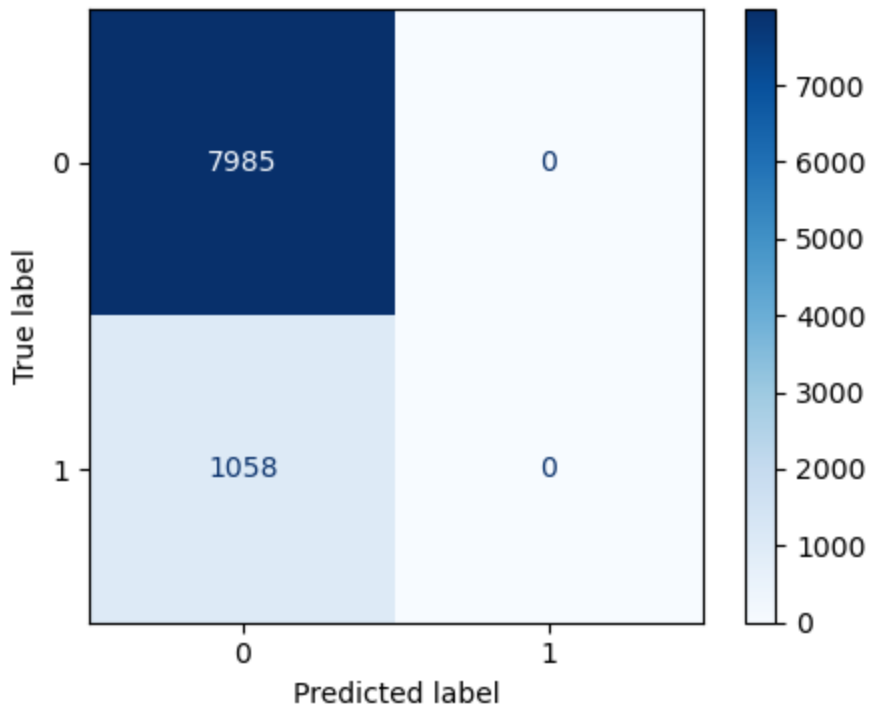
Conversion Rate (Precision@K): 0.0929
Lift@K:          0.79

```

DIAGNOSTIC REPORT (Threshold = 0.50):

	precision	recall	f1-score	support
0	0.8830	1.0000	0.9379	7985
1	0.0000	0.0000	0.0000	1058
accuracy			0.8830	9043
macro avg	0.4415	0.5000	0.4689	9043
weighted avg	0.7797	0.8830	0.8281	9043

Confusion Matrix - Baseline - Most Frequent



MODEL: Baseline - Stratified

GLOBAL METRICS:

ROC-AUC: 0.4970
Average Precision: 0.116415

RANKING METRICS (CAMPAIGN TRIAGE):

Base conversion rate: 0.1170

Top 1% clients:

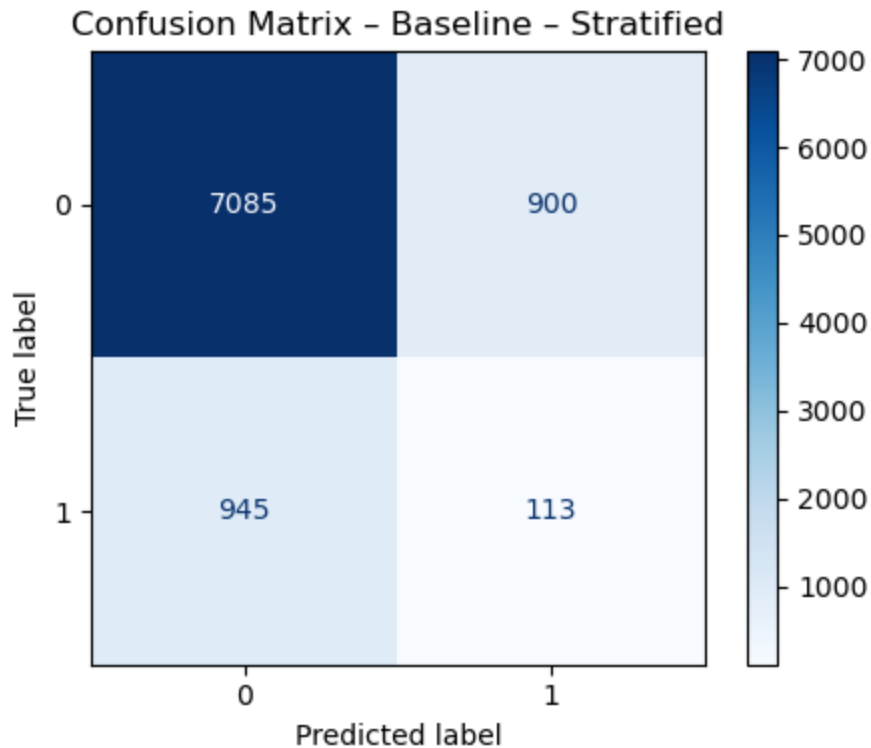
Conversion Rate (Precision@K): 0.1000
Lift@K: 0.85

Top 5% clients:

Conversion Rate (Precision@K): 0.1084
Lift@K: 0.93

DIAGNOSTIC REPORT (Threshold = 0.50):

	precision	recall	f1-score	support
0	0.8823	0.8873	0.8848	7985
1	0.1115	0.1068	0.1091	1058
accuracy			0.7960	9043
macro avg	0.4969	0.4970	0.4970	9043
weighted avg	0.7921	0.7960	0.7940	9043



=====

MODEL: Logistic Regression

=====

GLOBAL METRICS:

ROC-AUC: 0.7167
Average Precision: 0.264733

RANKING METRICS (CAMPAIGN TRIAGE):

Base conversion rate: 0.1170

Top 1% clients:

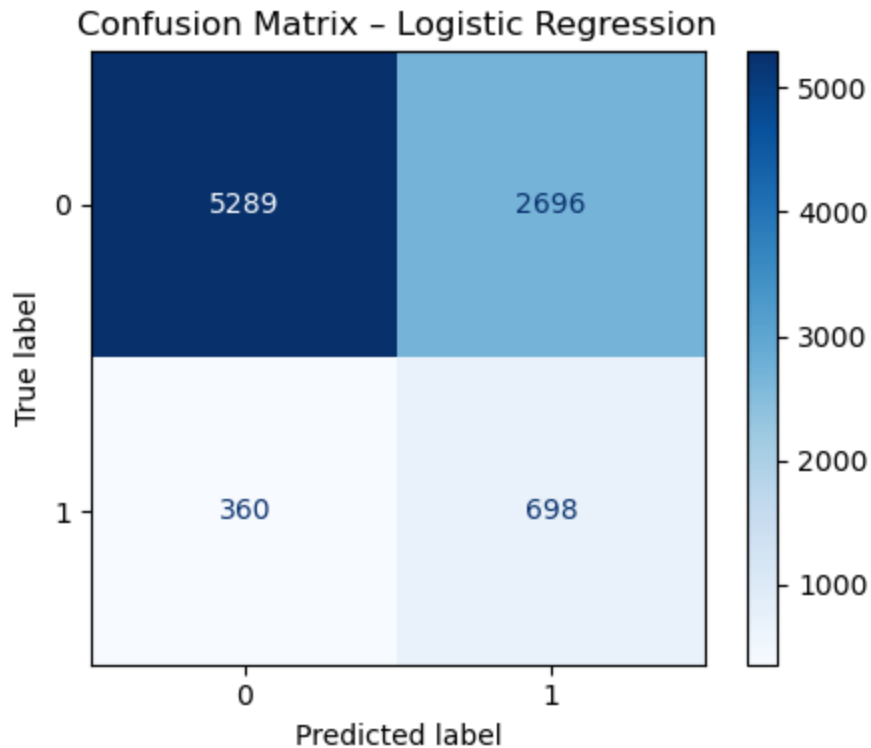
Conversion Rate (Precision@K): 0.4222
Lift@K: 3.61

Top 5% clients:

Conversion Rate (Precision@K): 0.3717
Lift@K: 3.18

DIAGNOSTIC REPORT (Threshold = 0.50):

	precision	recall	f1-score	support
0	0.9363	0.6624	0.7759	7985
1	0.2057	0.6597	0.3136	1058
accuracy			0.6621	9043
macro avg	0.5710	0.6611	0.5447	9043
weighted avg	0.8508	0.6621	0.7218	9043



=====

MODEL: Random Forest

=====

GLOBAL METRICS:

ROC-AUC: 0.7411
Average Precision: 0.331636

RANKING METRICS (CAMPAIGN TRIAGE):

Base conversion rate: 0.1170

Top 1% clients:

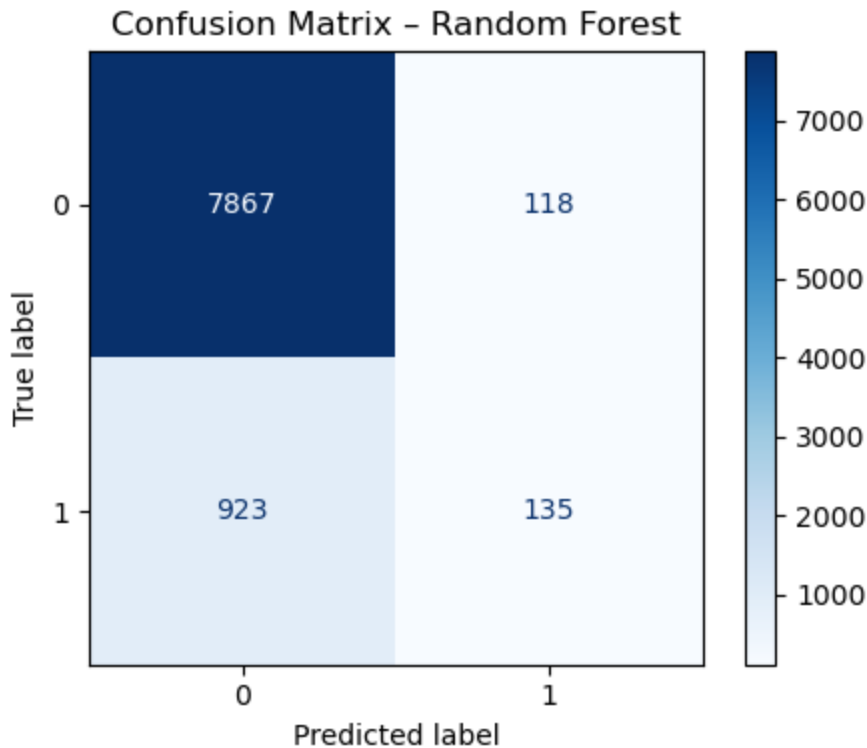
Conversion Rate (Precision@K): 0.5667
Lift@K: 4.84

Top 5% clients:

Conversion Rate (Precision@K): 0.4801
Lift@K: 4.10

DIAGNOSTIC REPORT (Threshold = 0.50):

	precision	recall	f1-score	support
0	0.8950	0.9852	0.9379	7985
1	0.5336	0.1276	0.2059	1058
accuracy			0.8849	9043
macro avg	0.7143	0.5564	0.5719	9043
weighted avg	0.8527	0.8849	0.8523	9043



=====

MODEL: XGBoost

=====

GLOBAL METRICS:

ROC-AUC: 0.7376
Average Precision: 0.353649

RANKING METRICS (CAMPAIGN TRIAGE):

Base conversion rate: 0.1170

Top 1% clients:

Conversion Rate (Precision@K): 0.6333
Lift@K: 5.41

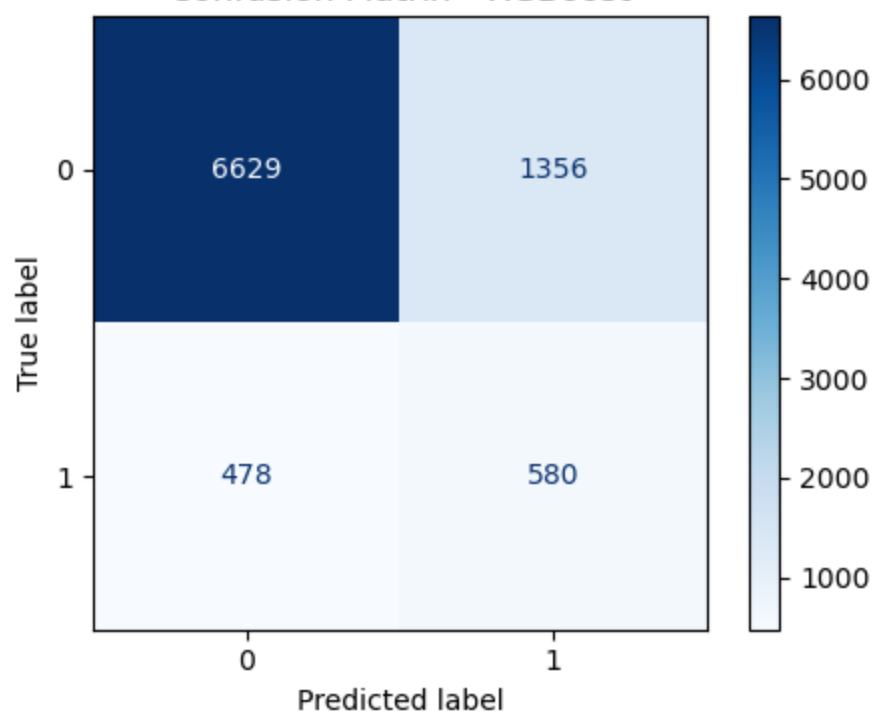
Top 5% clients:

Conversion Rate (Precision@K): 0.5221
Lift@K: 4.46

DIAGNOSTIC REPORT (Threshold = 0.50):

	precision	recall	f1-score	support
0	0.9327	0.8302	0.8785	7985
1	0.2996	0.5482	0.3874	1058
accuracy			0.7972	9043
macro avg	0.6162	0.6892	0.6330	9043
weighted avg	0.8587	0.7972	0.8210	9043

Confusion Matrix - XGBoost



```
[LightGBM] [Info] Number of positive: 4231, number of negative: 31937
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001977 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 709
[LightGBM] [Info] Number of data points in the train set: 36168, number of used features: 30
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.116982 -> initscore=-2.021327
[LightGBM] [Info] Start training from score -2.021327
```

```
=====
MODEL: LightGBM
=====
```

GLOBAL METRICS:

```
ROC-AUC:          0.7597
Average Precision: 0.378898
```

RANKING METRICS (CAMPAIGN TRIAGE):

```
Base conversion rate: 0.1170
```

Top 1% clients:

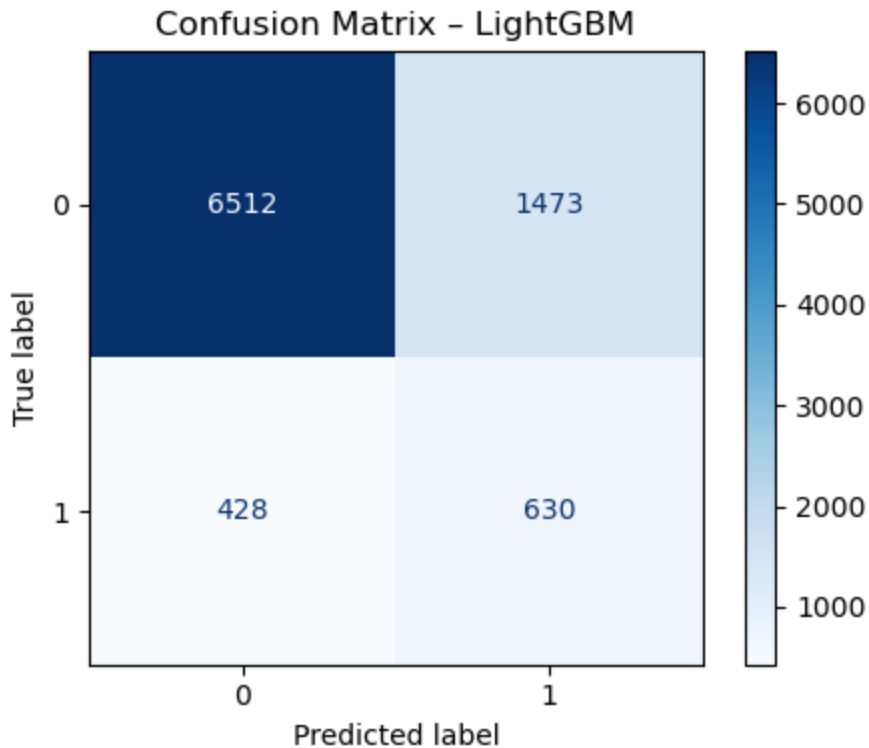
```
Conversion Rate (Precision@K): 0.6778
Lift@K:                        5.79
```

Top 5% clients:

```
Conversion Rate (Precision@K): 0.5398
Lift@K:                        4.61
```

DIAGNOSTIC REPORT (Threshold = 0.50):

	precision	recall	f1-score	support
0	0.9383	0.8155	0.8726	7985
1	0.2996	0.5955	0.3986	1058
accuracy			0.7898	9043
macro avg	0.6190	0.7055	0.6356	9043
weighted avg	0.8636	0.7898	0.8172	9043



```
Out[45]: {'Model': 'LightGBM',
          'ROC AUC': 0.7596611912932212,
          'Average Precision': 0.37889785327541337,
          'Base Rate': np.float64(0.11699657193409267),
          'Y Proba': array([0.21963534, 0.19363587, 0.25804673, ..., 0.37010323, 0.8359194
3,
          0.23801287], shape=(9043,))}
```

- Resultados

```
In [46]: def get_metrics_bank(
          model,
          X_test,
          y_test,
          ks=(0.01, 0.05)
        ):
    """
    Avaliação para COMPETIÇÃO DE MODELOS - BANK MARKETING

    - NÃO usa threshold fixo
    - Baseada em ranking probabilístico
    - Otimizada para priorização de contatos
    """

    # -----
    # Probabilidades da classe positiva (subscrição)
    # -----
    y_proba = model.predict_proba(X_test)[:, 1]

    # -----
    # Métricas globais (threshold-independente)
```

```

# -----
ap = average_precision_score(y_test, y_proba)
auc = roc_auc_score(y_test, y_proba)

metrics = {
    "AP": ap,
    "ROC_AUC": auc
}

# -----
# Métricas de ranking (campanha)
# -----

base_rate = y_test.mean()
order = np.argsort(y_proba)[::-1]

n = len(y_test)

for k in ks:
    cutoff = int(n * k)
    top_k_idx = order[:cutoff]

    # Taxa de conversão no Top-K
    precision_k = y_test.iloc[top_k_idx].mean()

    # Lift relativo ao contato aleatório
    lift_k = precision_k / base_rate if base_rate > 0 else np.nan

    metrics[f"Conversion@{int(k*100)}%"] = precision_k
    metrics[f"Lift@{int(k*100)}%"] = lift_k

return metrics

```

```

In [47]: results = []

models = {
    "Dummy - Most Frequent": dummy_pipe,
    "Dummy - Stratified": dummy_strat,
    "Logistic Regression": logreg_pipe,
    "Random Forest": rf_pipe,
    "XGBoost": xgb_pipe,
    "LGBM": lgbm_pipe
}

for name, model in models.items():
    metrics = get_metrics_bank(
        model,
        X_test,
        y_test_num,
        ks=(0.01, 0.05)
    )

    results.append([
        name,
        metrics["AP"],
        metrics["ROC_AUC"],
        metrics["Conversion@1%"],

```

```

        metrics["Lift@1%"],
        metrics["Conversion@5%"],
        metrics["Lift@5%"]
    ])

df_results = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Avg Precision",
        "ROC-AUC",
        "Conversion @ 1%",
        "Lift @ 1%",
        "Conversion @ 5%",
        "Lift @ 5%"
    ]
)

df_results.sort_values("Avg Precision", ascending=False, inplace=True)
df_results

```

Out[47]:

	Model	Avg Precision	ROC-AUC	Conversion @ 1%	Lift @ 1%	Conversion @ 5%	Lift @ 5%
5	LGBM	0.378898	0.759661	0.677778	5.793142	0.539823	4.614007
4	XGBoost	0.353649	0.737586	0.633333	5.413264	0.522124	4.462728
3	Random Forest	0.331636	0.741053	0.566667	4.843447	0.480088	4.103441
2	Logistic Regression	0.264733	0.716660	0.422222	3.608843	0.371681	3.176857
0	Dummy – Most Frequent	0.116997	0.500000	0.088889	0.759756	0.092920	0.794214
1	Dummy – Stratified	0.116415	0.497047	0.100000	0.854726	0.108407	0.926583

Modelos Promissores

- O modelo escolhido foi o XGBoost

```

In [48]: # Criar um diretório temporário para cache
cachedir = mkdtemp()

```

- LGBM

```

In [49]: lgbm_param_grid = {
    "model__max_depth": [3, 5, 7, 10, -1], # -1 = sem limite
    "model__num_leaves": [15, 31, 63, 127], # compatível com max_depth
    "model__min_child_samples": [10, 20, 50, 100],
    "model__learning_rate": [0.01, 0.05, 0.1],
}

```



```

    "model__subsample": [0.6, 0.8, 1.0],
    "model__colsample_bytree": [0.6, 0.8, 1.0],
    "model__reg_alpha": [0.0, 0.1, 0.5, 1.0],
    "model__reg_lambda": [0.5, 1.0, 2.0]
}

lgbm_pipe = Pipeline(
    steps=[
        ("preprocess", preprocessor),
        ("model", LGBMClassifier(
            objective="binary",
            scale_pos_weight=scale_pos_weight,
            random_state=42,
            n_jobs=-1,
            verbose=-1
        ))
    ],
    memory=cachedir
)

grid_lgbm = HalvingRandomSearchCV(
    estimator=lgbm_pipe,
    param_distributions=lgbm_param_grid,
    factor=3,
    resource="model__n_estimators",
    min_resources=100,
    max_resources=600,
    scoring="average_precision",
    cv=5,
    random_state=42,
    n_jobs=-1
)

print("Starting Search LightGBM...")
grid_lgbm.fit(X_train, y_train_num)

```

Starting Search LightGBM...

Out[49]:

▸ HalvingRandomSearchCV ⓘ ?

▸ best_estimator_: Pipeline

▸ preprocess: ColumnTransformer ⓘ

▸ num

▸ bin

▸ senior

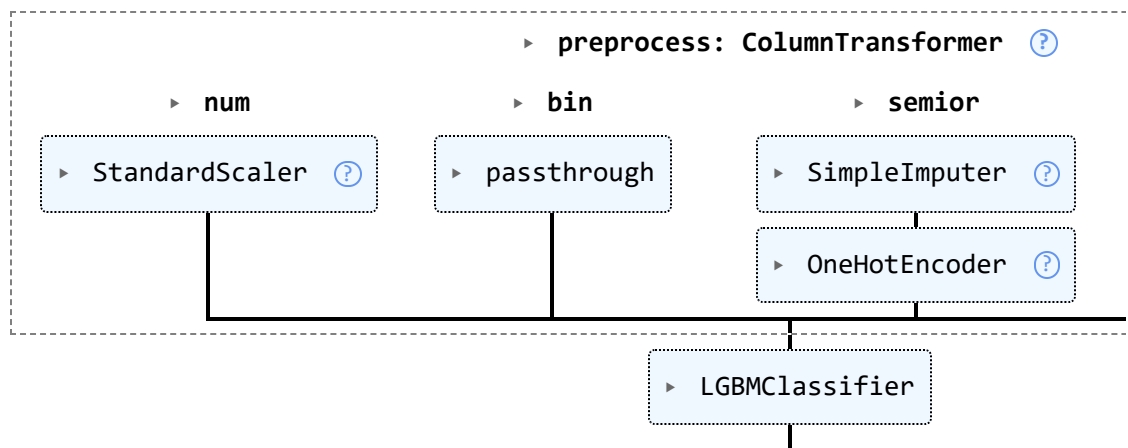
▸ StandardScaler ⓘ

▸ passthrough

▸ SimpleImputer ⓘ

▸ OneHotEncoder ⓘ

▸ LGBMClassifier



- XGBoost

```
In [50]: xgb_param_grid = {
    "model__max_depth": [3, 5, 7, 10],
    "model__min_child_weight": [1, 5, 10, 20],
    "model__learning_rate": [0.01, 0.05, 0.1],
    "model__subsample": [0.6, 0.8, 1.0],
    "model__colsample_bytree": [0.6, 0.8, 1.0],
    "model__reg_alpha": [0.0, 0.1, 0.5, 1.0],
    "model__reg_lambda": [0.5, 1.0, 2.0]
}

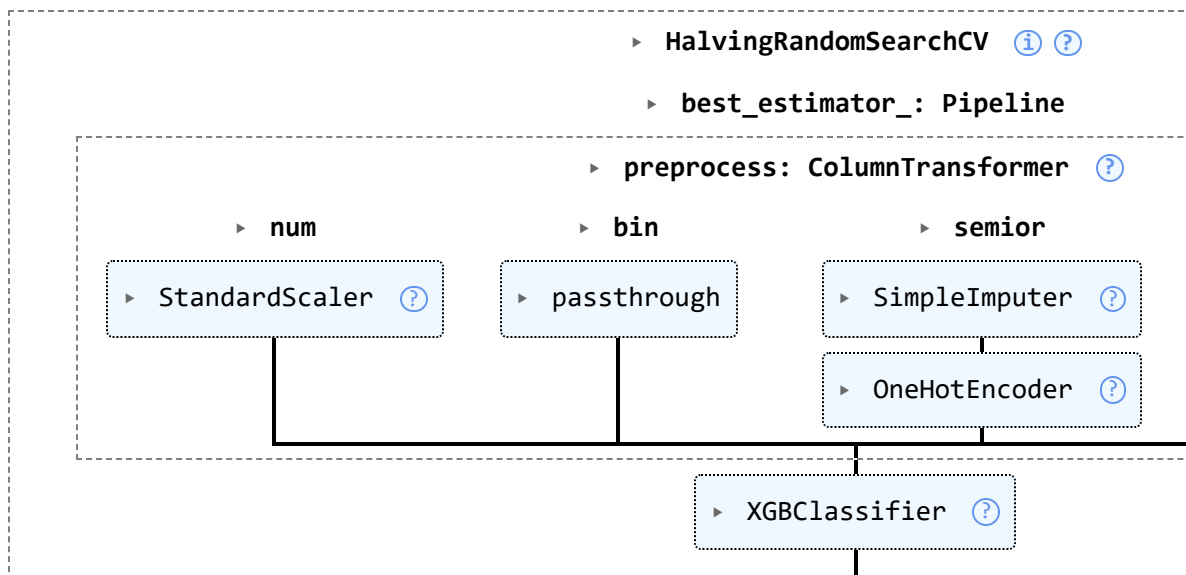
xgb_pipe = Pipeline(
    steps=[
        ("preprocess", preprocessor),
        ("model", XGBClassifier(
            objective="binary:logistic",
            eval_metric="logloss",
            scale_pos_weight=scale_pos_weight,
            random_state=42,
            verbosity=0,
            n_jobs=-1
        ))
    ],
    memory=cachedir
)

grid_xgb = HalvingRandomSearchCV(
    estimator=xgb_pipe,
    param_distributions=xgb_param_grid,
    factor=3,
    resource="model__n_estimators",
    min_resources=100,
    max_resources=600,
    scoring="average_precision",
    cv=5,
    random_state=42,
    n_jobs=-1
)

print("Starting Search XGBoost...")
grid_xgb.fit(X_train, y_train_num)
```

Starting Search XGBoost...

Out[50]:



- Comparação dos modelos promissores

```
In [51]: models_optimized = {
    "LGBM (Tuned)": grid_lgbm.best_estimator_,
    "XGBoost (Tuned)": grid_xgb.best_estimator_
}

results_opt = []

for name, model in models_optimized.items():
    metrics = get_metrics_bank(model, X_test, y_test_num)

    results_opt.append([
        name,
        metrics["AP"],
        metrics["ROC_AUC"],
        metrics["Conversion@1%"],
        metrics["Lift@1%"],
        metrics["Conversion@5%"],
        metrics["Lift@5%"]
    ])

df_results_opt = pd.DataFrame(
    results_opt,
    columns=[
        "Model",
        "Avg Precision (AP)",
        "ROC-AUC",
        "Conversion@1%",
        "Lift @ 1%",
        "Conversion@5%",
        "Lift @ 5%"
    ]
)
```

```
df_results_opt.sort_values("Avg Precision (AP)", ascending=False, inplace=True)
df_results_opt
```

Out[51]:

	Model	Avg Precision (AP)	ROC-AUC	Conversion@1%	Lift @ 1%	Conversion@5%	Lift @ 5%
1	XGBoost (Tuned)	0.381908	0.762846	0.688889	5.888112	0.546460	4.670737
0	LGBM (Tuned)	0.375367	0.756787	0.655556	5.603203	0.522124	4.462728

Análise de Erros e Interpretação do Modelo Final

- A seção utiliza técnicas avançadas de Inteligência Artificial Explicável (XAI) para detalhar o comportamento do modelo XGBoost, selecionado por sua performance superior em métricas de ranking. A análise destaca que o modelo não deve ser operado por um threshold fixo, mas sim por priorização probabilística (ranking), focando nos clientes com maior score para maximizar o retorno das campanhas. - Pontos Centrais da Interpretação: . Capacidade de Separação: O modelo demonstra uma separação parcial entre as classes; enquanto não-convertidos (Classe 0) concentram-se em probabilidades baixas, os convertidos (Classe 1) estão deslocados para a direita, embora haja uma sobreposição em zonas de incerteza; . Trade-off Operacional: A aplicação de um threshold diagnóstico de 0.20 permite atingir uma sensibilidade de 93,76%, identificando a grande maioria dos casos de conversão, ainda que à custa de um aumento nos falsos positivos; . Eficiência de Triagem (Lift): A estratégia de priorização mostra-se altamente eficaz, capturando 38% das conversões reais ao abordar apenas os top 10% dos clientes com maior score, o que representa um Lift de 3.8x em relação a contatos aleatórios; . Explicabilidade Local (SHAP): Através dos Force Plots, observa-se que perfis de alta probabilidade (99%) são impulsionados por fatores como um longo período desde o último contato (pdays = 515) e saldo positivo. Já a baixa probabilidade (0%) é dominada pela fadiga de contato, onde um número excessivo de ligações (campaign = 26) e saldos negativos destroem a chance de conversão.

- Análise de Thresholds (Ponto de Corte)

- Observação, maximizamos o recall para um threshold de 0.2, entretanto, a decisão operacional em marketing não se deve a este fator.

```
In [52]: # -----
# 1. Obter probabilidades para a classe positiva (Subscription)
# -----
y_proba = grid_xgb.best_estimator_.predict_proba(X_test)[: , 1]

# -----
# 2. Calcular precision e recall
# -----
precision, recall, thresholds = precision_recall_curve(y_test_num, y_proba)

# thresholds tem tamanho n-1, precision e recall tamanho n
thresholds_plot = np.append(thresholds, 1.0) # para alinhar os comprimentos (opcio

plt.figure(figsize=(12, 5))

# Plot Precision e Recall vs Threshold
plt.plot(thresholds_plot, precision, 'b--', label='Precision (Conversion Rate)', li
plt.plot(thresholds_plot, recall, 'r-', label='Recall (Coverage of conversion)', li
```

```

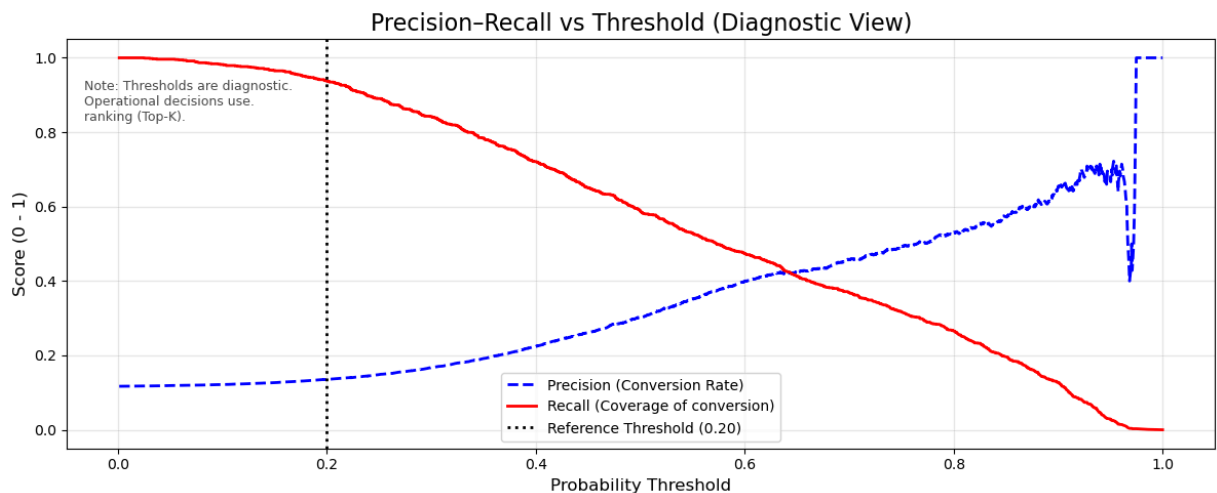
# -----
# 3. Marcar um threshold de referência para análise de trade-off
# -----
threshold_candidato = 0.2 # exemplo operacional
plt.axvline(threshold_candidato, color='black', linestyle=':',
            label=f'Reference Threshold ({threshold_candidato:.2f})', linewidth=2)

# -----
# 4. Ajustes de estética
# -----
plt.title('Precision-Recall vs Threshold (Diagnostic View)', fontsize=16)
plt.xlabel('Probability Threshold', fontsize=12)
plt.ylabel('Score (0 - 1)', fontsize=12)
plt.grid(alpha=0.3)
plt.legend(loc='best')

plt.text(
    0.015, 0.90,
    "Note: Thresholds are diagnostic.\nOperational decisions use.\nranking (Top-K).",
    transform=plt.gca().transAxes,
    fontsize=9,
    verticalalignment="top",
    alpha=0.7
)

plt.tight_layout()
plt.show()

```



- Probabilidades previstas e calibração

- Leitura do gráfico: . Classe 0 (azul): concentrada em probabilidades baixas; . Classe 1 (verde): deslocada à direita, mas com sobreposição; . Há separação parcial, não perfeita (esperado); . Threshold 0.5: Maior precisão, mas baixíssima cobertura (recall); . Threshold 0.2: Muito mais sensível, mas Aumenta FPs drasticamente.

```

In [53]: plt.figure(figsize=(12, 5))

# 1. Distribuição para os indivíduos não convertidos (Classe 0)
sns.kdeplot(
    y_proba[y_test_num == 0],
    fill=True,

```

```

        color="steelblue",
        label="Real: Non-converted (Class 0)",
        bw_adjust=0.7
    )

# 2. Distribuição para os indivíduos convertidos (Classe 1)
sns.kdeplot(
    y_proba[y_test_num == 1],
    fill=True,
    color="seagreen",
    label="Real: Converted (Class 1)",
    bw_adjust=0.7
)

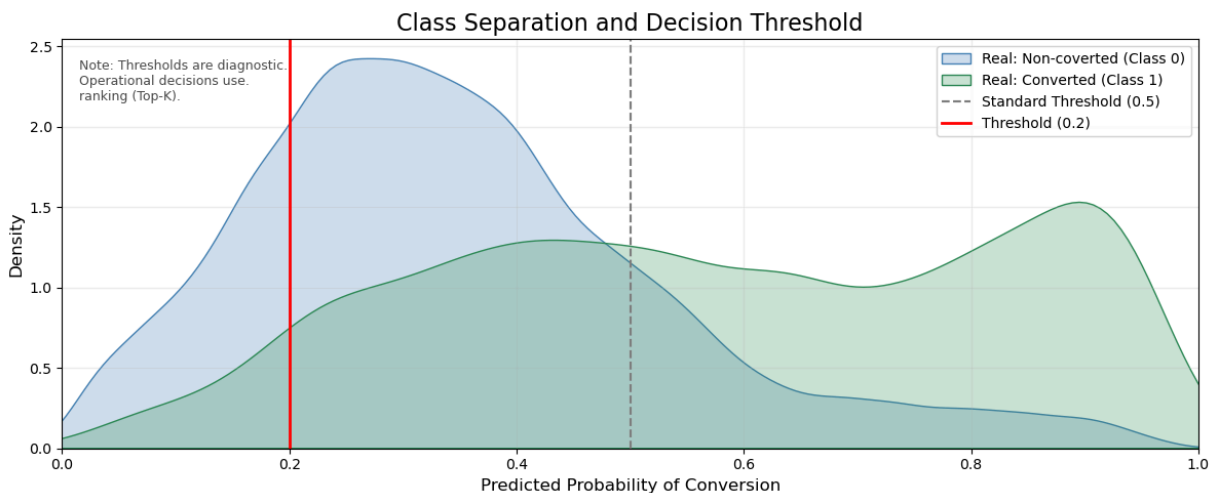
# 3. Linhas de Threshold para Comparação
plt.axvline(0.5, color="gray", linestyle="--", label="Standard Threshold (0.5)")
plt.axvline(0.2, color="red", linestyle="-", linewidth=2, label="Threshold (0.2)")

# 4. Formatação Técnica
plt.title("Class Separation and Decision Threshold", fontsize=16)
plt.xlabel("Predicted Probability of Conversion", fontsize=12)
plt.ylabel("Density", fontsize=12)
plt.xlim(0, 1)
plt.legend(loc='best')
plt.grid(alpha=0.2)

plt.text(
    0.015, 0.95,
    "Note: Thresholds are diagnostic.\nOperational decisions use.\nraking (Top-K).",
    transform=plt.gca().transAxes,
    fontsize=9,
    verticalalignment="top",
    alpha=0.7
)

plt.tight_layout()
plt.show()

```



- Calibração

- O modelo é conservador ao estar consistentemente abaixo da reta de 45 graus.

```
In [54]: # 1. Calcular a Curva de Calibração
# n_bins=10 divide as probabilidades em fatias de 10%
prob_true, prob_pred = calibration_curve(y_test_num, y_proba, n_bins=10)

# 2. Calcular Brier Score (Quanto menor, melhor a calibração)
brier = brier_score_loss(y_test_num, y_proba)
print(f"Brier Score (Conversion): {brier:.4f}")

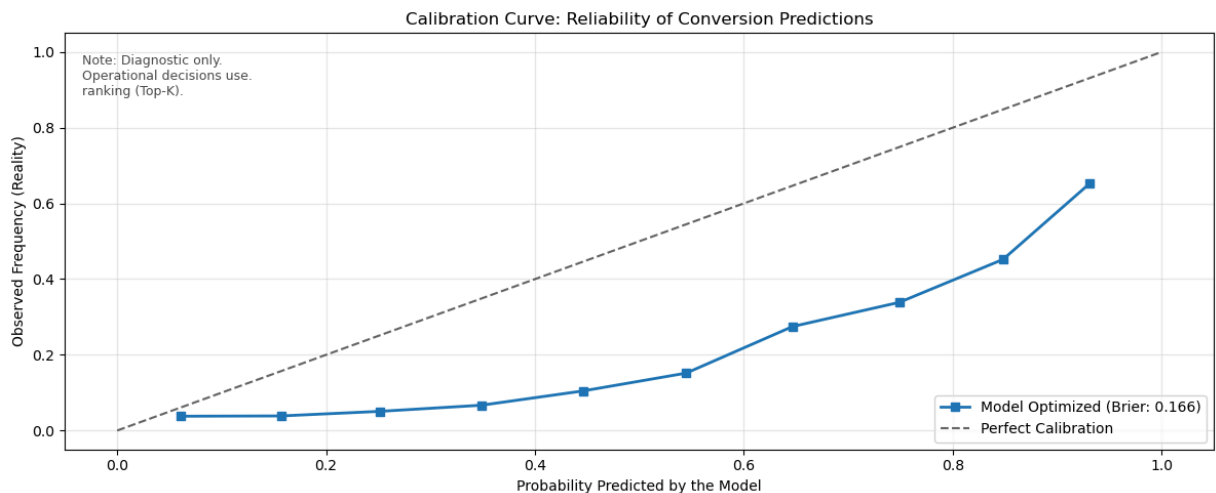
# 3. Plotagem
plt.figure(figsize=(12, 5))
plt.plot(prob_pred, prob_true, marker='s', linewidth=2, label=f'Model Optimized (Br
plt.plot([0, 1], [0, 1], linestyle='--', color='black', alpha=0.6, label='Perfect C

plt.xlabel('Probability Predicted by the Model')
plt.ylabel('Observed Frequency (Reality)')
plt.title('Calibration Curve: Reliability of Conversion Predictions')
plt.legend(loc='best')
plt.grid(alpha=0.3)

plt.text(
    0.015, 0.95,
    "Note: Diagnostic only.\nOperational decisions use.\nranking (Top-K).",
    transform=plt.gca().transAxes,
    fontsize=9,
    verticalalignment="top",
    alpha=0.7
)

plt.tight_layout()
plt.show()
```

Brier Score (Conversion): 0.1657



- Curvas ROC-AUC

- A inclusão da curva ROC-AUC é válida para o relatório, desde que acompanhada da ressalva técnica correta. Em um dataset com um pouco mais de 11% de convertidos, a ROC-AUC pode ser considerada "otimista" ou até enganosa, pois ela avalia o equilíbrio entre a Taxa de Verdadeiros Positivos (Recall) e a Taxa de Falsos Positivos.

```
In [55]: # Curva ROC
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

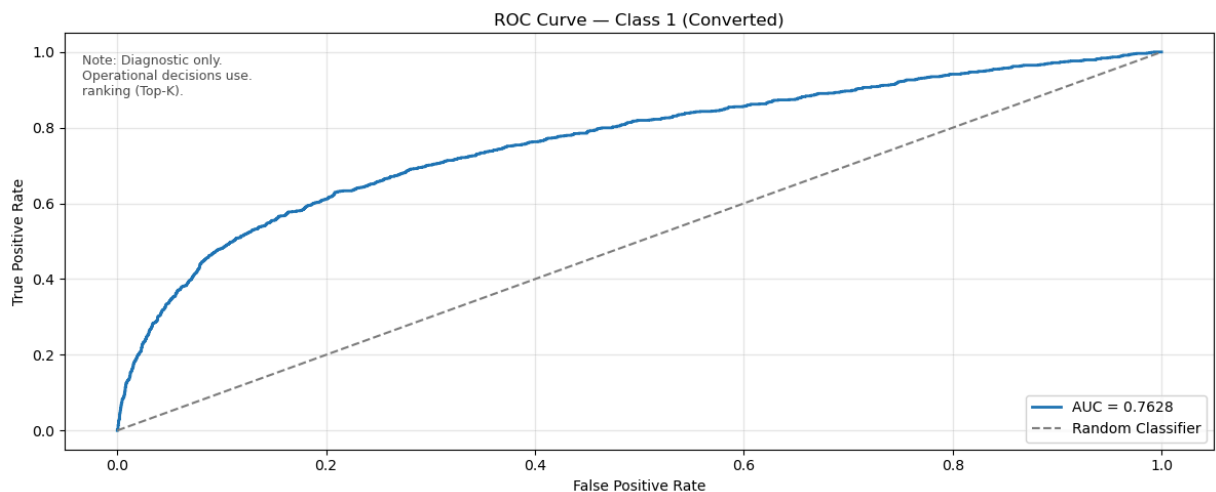
# AUC
auc = roc_auc_score(y_test, y_proba)

plt.figure(figsize=(12, 5))
plt.plot(fpr, tpr, lw=2, label=f"AUC = {auc:.4f}")
plt.plot([0, 1], [0, 1], "--", color="gray", label="Random Classifier")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve – Class 1 (Converted)")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)

plt.text(
    0.015, 0.95,
    "Note: Diagnostic only.\nOperational decisions use.\nranking (Top-K).",
    transform=plt.gca().transAxes,
    fontsize=9,
    verticalalignment="top",
    alpha=0.7
)

plt.tight_layout()
plt.show()
```



- Matriz de confusão

- Com o threshold de 0.2, o modelo atingiu uma Sensibilidade (Conversão) de 93.76%, identificando 992 casos de diabetes e perdendo apenas 66 (Falsos Negativos).

```
In [56]: # 1. Aplicar o Threshold de 0.20 (definido nas análises anteriores)
# Isso transforma probabilidades em classes (0 ou 1)
y_pred_clinico = (y_proba >= 0.20).astype(int)

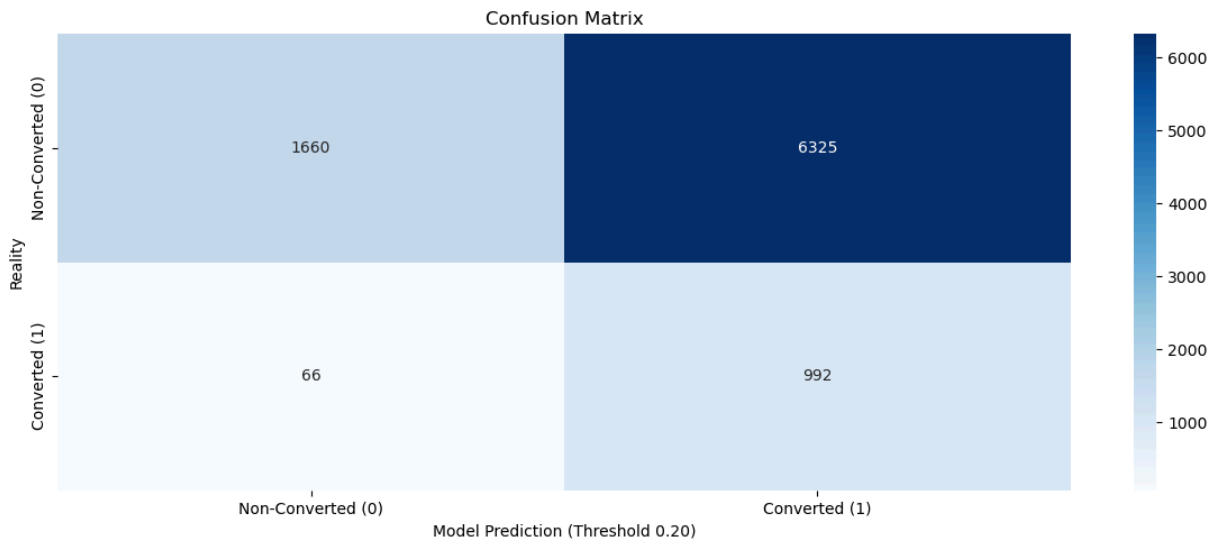
# 2. Gerar a Matriz de Confusão
cm = confusion_matrix(y_test_num, y_pred_clinico)
```



```
# 3. Plotagem
plt.figure(figsize=(12, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=['Non-Converted (0)', 'Converted (1)'],
            yticklabels=['Non-Converted (0)', 'Converted (1)'])

plt.xlabel("Model Prediction (Threshold 0.20)")
plt.ylabel("Reality")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

# 4. Extração de Métricas de Negócio
tn, fp, fn, tp = cm.ravel()
recall = tp / (tp + fn)
print(f"Sensitivity (Conversion): {recall:.2%}")
print(f"Conversion Cases Identified: {tp}")
print(f"Missed Conversion Cases (False Negatives): {fn}")
```



Sensitivity (Conversion): 93.76%
 Conversion Cases Identified: 992
 Missed Conversion Cases (False Negatives): 66

- Eficiência de triagem

- Insights: . Ao abordar apenas 10% dos clientes com maior score, é capturado ~ 38% de todas as conversões reais, com uma precisão de ~ 44%; . Com 20% da população, é capturado mais da metade de todas as conversões, mantendo um lift ainda alto; . A partir de 30–40% da população, o modelo continua agregando valor, mas o ganho marginal por cliente cai rapidamente; . Acima de 60% da população, o retorno (lift) se aproxima do acaso, então, recomenda-se que não opere campanhas acima desse limite.

```
In [57]: df_triage = pd.DataFrame({
        "y_true": y_test_num.reset_index(drop=True),
        "y_proba": y_proba
    })

# Ordenar do maior risco / propensão para o menor
df_triage = df_triage.sort_values(
    "y_proba", ascending=False
).reset_index(drop=True)
```

```

# Criar decis baseados no RANKING (não no valor da probabilidade)
df_triage["decile"] = pd.qcut(
    np.arange(len(df_triage)),
    q=10,
    labels=np.arange(1, 11)
)

total_positives = df_triage["y_true"].sum()
base_rate = total_positives / len(df_triage)

gains_table = (
    df_triage
    .groupby("decile", observed=True)
    .agg(
        population=("y_true", "count"),
        positives=("y_true", "sum")
    )
    .reset_index()
)

# Cálculos acumulados
gains_table["cum_population"] = gains_table["population"].cumsum()
gains_table["cum_positives"] = gains_table["positives"].cumsum()

gains_table["population_%"] = gains_table["cum_population"] / len(df_triage) * 100
gains_table["cumulative_gain_%"] = gains_table["cum_positives"] / total_positives *

# Lift acumulado (métrica estratégica)
gains_table["cumulative_lift"] = (
    gains_table["cumulative_gain_%"] / gains_table["population_%"]
)

# Precision acumulada (opcional, mas muito útil)
gains_table["precision_at_k"] = (
    gains_table["cum_positives"] / gains_table["cum_population"]
)

display(
    gains_table[
        [
            "decile",
            "population_%",
            "cumulative_gain_%",
            "cumulative_lift",
            "precision_at_k"
        ]
    ].round(3)
)

plt.figure(figsize=(12, 5))

plt.plot(
    gains_table["population_%"],
    gains_table["cumulative_gain_%"],
    marker="o",
    linewidth=2,

```

```

        label="Model (Ranked Screening)"
    )

    # Linha aleatória (baseline)
    plt.plot(
        [0, 100],
        [0, 100],
        linestyle="--",
        color="gray",
        label="Random Selection"
    )

    plt.title("Cumulative Gain Chart - Screening / Marketing", fontsize=14)
    plt.xlabel("Percentage of Population Examined (Highest Score First)", fontsize=12)
    plt.ylabel("Percentage of Positives Captured", fontsize=12)

    # Destaque operacional (Top 10%)
    top10 = gains_table.iloc[0]

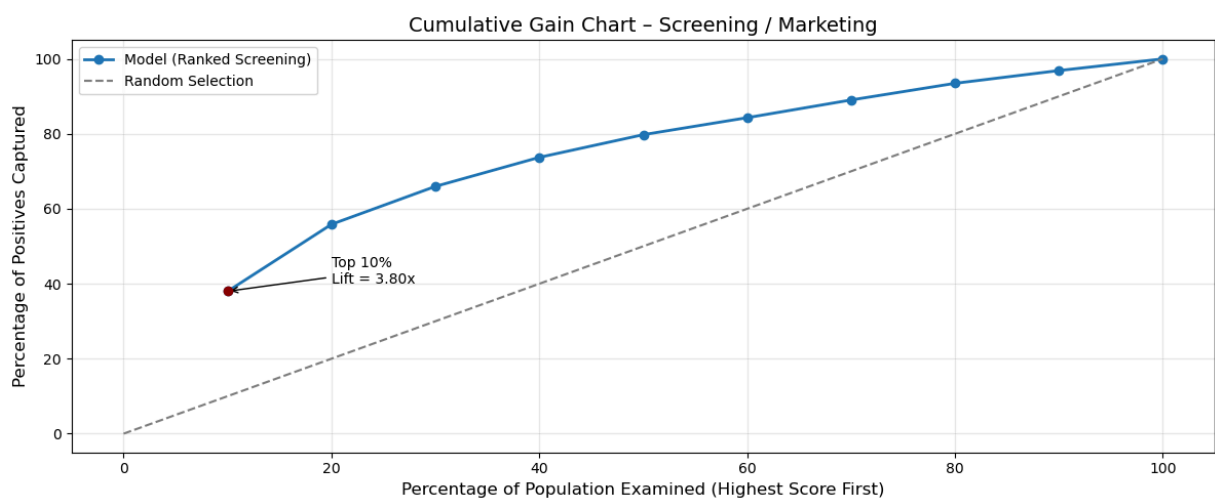
    plt.scatter(
        top10["population_%"],
        top10["cumulative_gain_%"],
        color="darkred",
        zorder=5
    )

    plt.annotate(
        f"Top 10%\nLift = {top10['cumulative_lift']:.2f}x",
        xy=(top10["population_%"], top10["cumulative_gain_%"]),
        xytext=(20, 40),
        arrowprops=dict(arrowstyle="->")
    )

    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()

```

	decile	population_%	cumulative_gain_%	cumulative_lift	precision_at_k
0	1	10.008	37.996	3.797	0.444
1	2	20.004	55.860	2.792	0.327
2	3	30.001	65.974	2.199	0.257
3	4	39.998	73.724	1.843	0.216
4	5	50.006	79.773	1.595	0.187
5	6	60.002	84.310	1.405	0.164
6	7	69.999	89.036	1.272	0.149
7	8	79.996	93.478	1.169	0.137
8	9	89.992	96.881	1.077	0.126
9	10	100.000	100.000	1.000	0.117



- Shap summary

- As variáveis de maior impacto SHAP estão relacionadas a campanhas anteriores, idade e empréstimo habitacional.

```
In [63]: # Recuperar componentes do pipeline (ajuste o nome do seu pipeline se necessário)
best_pipeline = grid_xgb.best_estimator_
classifier_model = best_pipeline.named_steps['model']
preprocessor_step = best_pipeline.named_steps['preprocess']

# 1. Criar o df_test_transformed (Obrigatório para o SHAP e para a filtragem)
X_test_transformed = preprocessor_step.transform(X_test)
feature_names = preprocessor_step.get_feature_names_out()
df_test_transformed = pd.DataFrame(X_test_transformed, columns=feature_names)

# 2. Inicializar o Explainer e calcular SHAP
explainer = shap.TreeExplainer(classifier_model)
# O parâmetro check_additivity=False é útil se houver pequenas discrepâncias de flo
shap_values = explainer.shap_values(df_test_transformed, check_additivity=False)

# 3. Tratamento para garantir a classe positiva (1) se for uma lista
```

```

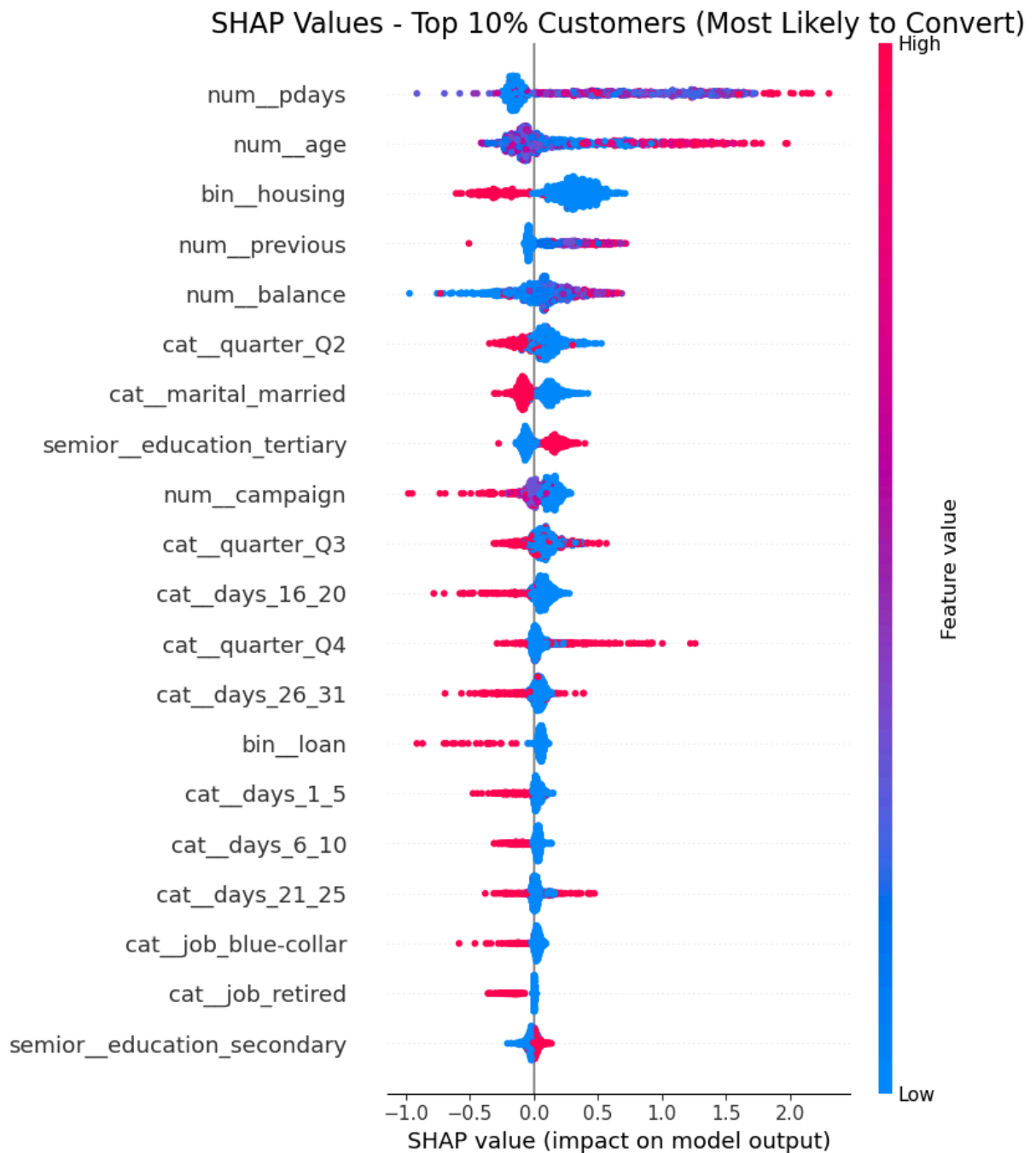
if isinstance(shap_values, list):
    shap_values_target_full = shap_values[1]
elif len(shap_values.shape) == 3:
    shap_values_target_full = shap_values[:, :, 1]
else:
    shap_values_target_full = shap_values

# 4. Identificar o Top 10%
y_probs = best_pipeline.predict_proba(X_test)[:, 1]
threshold_10_percent = np.percentile(y_probs, 90)
top_10_indices = np.where(y_probs >= threshold_10_percent)[0]

# 5. Filtrar dados e valores SHAP usando os índices
df_test_top_10 = df_test_transformed.iloc[top_10_indices]
shap_values_top_10 = shap_values_target_full[top_10_indices]

# 6. Plotagem final
plt.figure(figsize=(12, 10))
plt.title(f"SHAP Values - Top 10% Customers (Most Likely to Convert)", fontsize=16)
shap.summary_plot(
    shap_values_top_10,
    df_test_top_10,
    show=False
)
plt.tight_layout()
plt.show()

```



- Shap dependence plot

- Em média, indivíduos entre 60–80 anos que não são aposentados apresentam maior contribuição positiva ao score de conversão, especialmente quando não possuem empréstimo habitacional e apresentam valores elevados de pdays, com efeitos sazonais moderados

```
In [64]: # =====
# 1. PREPARAÇÃO DOS DADOS TOTAIS (GARANTINDO ALINHAMENTO)
# =====

# Transformar X_test completo e garantir nomes de colunas
X_test_transformed = preprocessor_step.transform(X_test)
feature_names = preprocessor_step.get_feature_names_out()
df_test_transformed = pd.DataFrame(X_test_transformed, columns=feature_names)
```

```

# Criar base de exibição com nomes limpos e valores reais (para o eixo X do gráfico)
X_display = df_test_transformed.copy()
X_display.columns = [col.split('__')[-1] for col in X_display.columns]

# Restaurar valores originais para colunas numéricas (Age, Duration, etc)
cols_numericas_reais = ['duration', 'age', 'balance', 'campaign', 'pdays']
for col in cols_numericas_reais:
    if col in X_display.columns:
        X_display[col] = X_test[col].values
    else:
        # Busca flexível caso o nome tenha mudado no preprocessor
        cols_match = [c for c in X_display.columns if col in c]
        if cols_match:
            X_display[cols_match[0]] = X_test[col].values

# =====
# 2. CÁLCULO DO SHAP PARA A BASE COMPLETA
# =====

print(f"SHAP Calculated for {len(df_test_transformed)} records...")
explainer = shap.TreeExplainer(classifier_model)
shap_values_raw = explainer.shap_values(df_test_transformed, check_additivity=False)

# Tratamento para garantir a classe positiva (1)
if isinstance(shap_values_raw, list):
    shap_values_target = shap_values_raw[1]
elif len(shap_values_raw.shape) == 3:
    shap_values_target = shap_values_raw[:, :, 1]
else:
    shap_values_target = shap_values_raw

# =====
# 3. FILTRAGEM DO TOP K@10% (OS MELHORES CLIENTES)
# =====

# Obter probabilidades de conversão
y_probs = best_pipeline.predict_proba(X_test)[:, 1]

# Definir threshold do Top 10% (Percentil 90)
threshold_10_percent = np.percentile(y_probs, 90)
top_10_indices = np.where(y_probs >= threshold_10_percent)[0]

# Aplicar o filtro (Corrigido para top_10)
X_display_top10 = X_display.iloc[top_10_indices]
shap_values_top10 = shap_values_target[top_10_indices]

print(f"Sucesso! Analisando {len(top_10_indices)} clientes com maior score.")

# =====
# 4. IDENTIFICAÇÃO E PLOTAGEM DAS TOP FEATURES NO TOP 10%
# =====

# Calcular importância Local (Média absoluta dos valores SHAP apenas no Top 10%)
local_importance = np.abs(shap_values_top10).mean(0)
feature_imp_df = pd.DataFrame({
    'feature': X_display_top10.columns,

```

```

    'importance': local_importance
}).sort_values(by='importance', ascending=False)

# Selecionar as 3 features que mais influenciam o Top 10%
top_3_features = feature_imp_df['feature'].head(3).tolist()

# Plotagem
n_features = len(top_3_features)
fig, axes = plt.subplots(nrows=1, ncols=n_features, figsize=(6*n_features, 5))

# Garantir que axes seja iterável mesmo com 1 feature
if n_features == 1:
    axes = [axes]

for i, feature_name in enumerate(top_3_features):
    shap.dependence_plot(
        feature_name,
        shap_values_top10,
        X_display_top10,
        interaction_index="auto",
        ax=axes[i],
        show=False,
        alpha=0.7,
        dot_size=30
    )
    axes[i].set_title(f"Top 10% Impact: {feature_name}", fontsize=12, fontweight='b')
    axes[i].grid(True, linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

```

SHAP Calculated for 9043 records...

Sucesso! Analisando 905 clientes com maior score.



- Shap force plot

- Perfil de Máxima Probabilidade ($f(x) = 0.99$): Este cliente tem 99% de chance de conversão. O gráfico mostra um "cabo de guerra" onde as forças vermelhas (que aumentam a probabilidade) esmagaram as forças azuis: . Pdays = 515: Esta é a variável com maior impacto positivo (seta vermelha mais larga). Isso indica que o cliente foi contatado há muito tempo em uma campanha anterior; . Quarter_Q4 = 1: O fato de estarmos no quarto trimestre impacta positivamente a conversão; . Days_26_31 = 1: O fato de estarmos entre os dias 26 e 31 impacta positivamente a conversão; . Balance = 992: Um saldo positivo de quase mil euros. . Housing = 1 (Força Azul): Ter um empréstimo imobiliário é o único fator relevante tentando reduzir a chance de conversão, mas seu impacto é pequeno comparado aos outros; - Perfil de Mínima Probabilidade ($f(x) = 0.00$): Este cliente tem quase 0% de chance de conversão. Aqui, as forças azuis dominam completamente: . Campaign = 26: Este é o maior "vilão" da conversão. O cliente foi contatado 26 vezes nesta campanha. Isso geralmente indica fadiga do cliente ou que ele é um "não" irredutível; quanto mais você liga, menor a chance de ele converter; . Balance = -160: O saldo negativo (conta no vermelho) é um forte impeditivo para a

aquisição de produtos bancários; . Quarter_Q2 = 1: O fato de estarmos no segundo trimestre impacta negativamente a conversão; . Job_blue-collar = 1: O tipo de profissão está atuando como um redutor de probabilidade neste modelo específico; . Age = 45: Nesta faixa etária e com essas condições financeiras, o modelo vê baixíssima propensão.

```
In [66]: # =====
# 1. CRIAR O DATAFRAME DE EXIBIÇÃO (X_display) - BASE COMPLETA
# =====
# Usamos o DataFrame completo transformado anteriormente
X_display = df_test_transformed.copy()

# A. Limpar os nomes das colunas
X_display.columns = [col.split('__')[-1] for col in X_display.columns]

# B. Injetar os valores REAIS (Alinhado com X_test completo)
cols_reais = ['age', 'job', 'marital', 'education',
              'default', 'balance', 'housing',
              'loan', 'campaign', 'pdays', 'previous', 'days', 'quarter']

X_raw_full = X_test.copy()

for col in cols_reais:
    if col in X_display.columns:
        X_display[col] = X_raw_full[col].values

# =====
# 2. CÁLCULOS DO SHAP E SELEÇÃO DA CLASSE 1 (XGBOOST)
# =====

# A. Ajuste do Valor Base (Expected Value)
# Para XGBoost Binário, o expected_value costuma ser um escalar único (em Log-odds)
base_value = explainer.expected_value

# Se por algum motivo o explainer retornar uma lista (comum em certas versões/confi
if isinstance(base_value, (list, np.ndarray)) and len(base_value) > 1:
    base_value = base_value[1]

print(f"Valor Base do XGBoost (Log-Odds): {base_value:.4f}")

# B. Garantir SHAP Values da Classe 1
# 'shap_values_target' já foi definido no passo anterior como a matriz da Classe 1
shap_vals_target = shap_values_target

# C. Encontrar índices extremos na BASE COMPLETA
# Calculamos a predição final somando o base_value aos impactos SHAP
total_predictions = shap_vals_target.sum(axis=1) + base_value

idx_high = np.argmax(total_predictions) # Posição do cliente "Mais Provável"
idx_low = np.argmin(total_predictions) # Posição do cliente "Menos Provável"

print(f"High Probability Client (Posição {idx_high}): Log-odds ≈ {total_predictions[idx_high]:.4f}")
print(f"Low Probability Client (Posição {idx_low}): Log-odds ≈ {total_predictions[idx_low]:.4f}")

# =====
# 3. GERAR FORCE PLOTS INTERATIVOS (LOGIT LINK)
# =====

def plot_force_xgb(idx_pos, title):
    print(f"\n--- {title} ---")
```

```
# Inicializa JS no notebook
shap.initjs()
```

```
# Link='logit' converte de Log-odds para probabilidade real (0 a 1)
# Isso é essencial para tornar o gráfico interpretável para humanos
display(shap.force_plot(
    base_value,
    shap_vals_target[idx_pos],
    X_display.iloc[idx_pos, :],
    link="logit",
    matplotlib=False
))
```

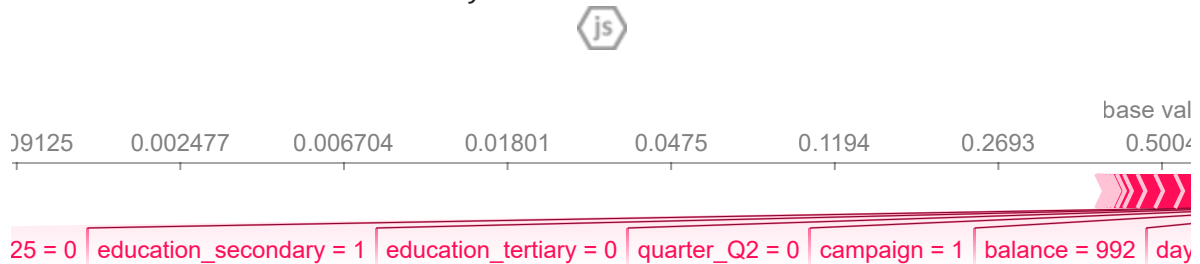
```
# Gerar os Gráficos de "Ato de Fé" (Alta Probabilidade) e "Rejeição" (Baixa Probabi
plot_force_xgb(idx_high, "Profile: Maximum Probability of Conversion")
plot_force_xgb(idx_low, "Profile: Lowest Probability of Conversion")
```

Valor Base do XGBoost (Log-Odds): 0.0016

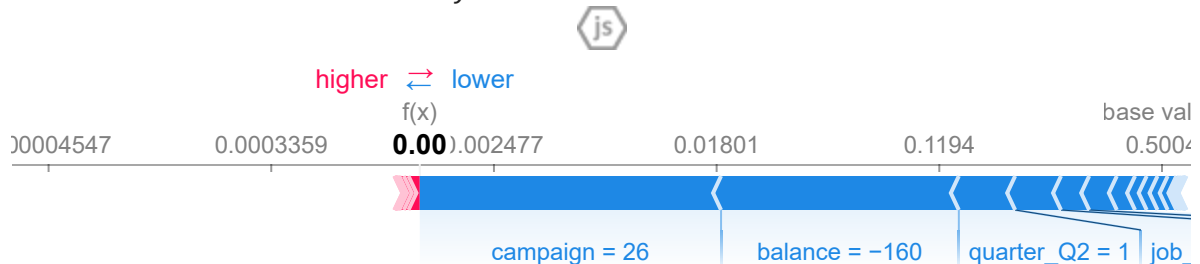
High Probability Client (Posição 2604): Log-odds ≈ 4.29

Low Probability Client (Posição 5851): Log-odds ≈ -6.66

--- Profile: Maximum Probability of Conversion ---



--- Profile: Lowest Probability of Conversion ---



Salvando o Modelo

```
In [67]: # 1. Extrair o pipeline vencedor (Pré-processamento + Modelo)
modelo_final = grid_xgb.best_estimator_

# 2. Salvar em um arquivo .pkl
joblib.dump(modelo_final, 'modelo_bank_marketing_xgboost.pkl')
```

```
Out[67]: ['modelo_bank_marketing_xgboost.pkl']
```

Síntese

- **Objetivos:** . O objetivo central deste projeto foi desenvolver um modelo de alta performance para prever a subscrição de depósitos a prazo em uma instituição bancária. Buscou-se ir além da classificação binária tradicional, focando na priorização de contatos por meio de um ranking probabilístico, visando otimizar a eficiência das campanhas de marketing direto. - **Metodologia Resumida:** . **Tratamento de Dados:** Realizou-se a limpeza de valores faltantes em variáveis como job e education via imputação por moda, além da remoção crítica de variáveis causadoras de data leakage (duration, contact e poutcome); . **Engenharia de Features:** Variáveis temporais foram reagrupadas em trimestres e faixas de dias para reduzir a alta cardinalidade e capturar padrões sazonais; . **Modelagem:** Foram testados algoritmos como Regressão Logística, Random Forest, XGBoost e LightGBM. O modelo XGBoost foi selecionado após tunagem de hiperparâmetros por apresentar o melhor equilíbrio entre Precisão Média (AP) e Lift; . **Avaliação:** Utilizou-se uma abordagem baseada em ranking (Precision@K e Lift@K) em vez de um threshold fixo, simulando a realidade de uma equipe de marketing. - **Resultados e Validação de Hipóteses:** . **Desempenho Geral:** O modelo final (XGBoost Tuned) atingiu um Lift de 5.89x no Top 1% e 4.67x no Top 5%. Isso significa que, ao focar nos 10% melhores clientes da lista, o banco captura cerca de 38% de todas as conversões reais; . **H1 (Histórico):** Embora o sucesso anterior seja informativo, a variável poutcome foi removida por ser considerada data leakage operacional, impedindo sua validação como preditor futuro ético; . **H2 (Saúde Financeira):** O saldo positivo (balance) apareceu como fator de impacto positivo no SHAP para o Top 10% de clientes; . **H3 (Fadiga de Contato):** O SHAP indicou que números excessivos de contatos (campaign = 26) são o maior impeditivo para a conversão; . **H4 (Perfil Demográfico):** Clientes entre 60 e 80 anos apresentaram maior contribuição positiva ao score de conversão no modelo final; . **H5 (Barreiras de Crédito):** Ter um empréstimo habitacional (housing) foi associado como um redutor consistente da probabilidade de conversão. - **Limitações:** . **Desbalanceamento:** O dataset possui apenas 11,7% de casos positivos, o que torna o modelo conservador em suas previsões de probabilidade absoluta; . **Viés de Seleção:** As variáveis contact e poutcome apresentaram ausência não aleatória, o que forçou a sua remoção para evitar viés operacional, mas resultou na perda de informação histórica potencialmente útil; . **Variáveis Omitidas:** A falta de dados sobre a renda mensal exata ou o patrimônio total do cliente limita a análise da capacidade real de investimento. - **Sugestões para Trabalhos Futuros:** . **Enriquecimento de Dados:** Integrar dados de comportamento digital (acessos ao app) para complementar os dados de chamadas telefônicas; . **Modelagem de Uplift:** Migrar da previsão de propensão para a modelagem de Uplift, visando identificar apenas os clientes que converteriam por causa da ligação; . **Monitoramento Streamlit:** Implementar um módulo de feedback loop na aplicação Streamlit, onde o vendedor registra o desfecho real da ligação para retreinar o modelo periodicamente; . **Calibração de Probabilidade:** Testar técnicas de calibração (como Isotonic Regression) para aproximar as probabilidades previstas da realidade observada, facilitando análises de ROI.